

Kirstine-Amalie, Henrik, Nicolai H. og Christian F.

Datamatiker - DMVE251

UCL Erhvervsakademi og Professionshøjskole
Boulevarden 25, 7100 Vejle

<https://github.com/hiha76740/SecondSemesterEksamensProjekt>



29. Maj 2026

Antal anslag: 74.928

Indholdsfortegnelse

1. Indledning.....	4
1.1 Præsentation af BookRight Klinik & Wellness ApS.....	4
1.2 Problemstilling	4
1.3 Projektets formål	4
2. Problemformulering og afgrænsning	5
2.1 Problemformulering.....	5
2.2 Underspørgsmål.....	5
2.3 Afgrænsning	6
3. Metode	6
3.1 Modificeret Scrum som systemudviklingsmetode	6
3.2 Samarbejde og projektorganisering.....	7
3.3 Analyse-, krav- og dokumentationsmetoder.....	8
3.4 Digitale hjælpевærktøjer og generativ AI	9
4. Forretningsanalyse.....	10
4.1 As-Is-analyse: Nuværende arbejdsproces	10
4.2 To-Be-analyse: Fremtidig arbejdsproces med BookRight-systemet.....	11
4.3 SWOT-analyse	12
4.3.1 Uddybning af SWOT-analysen	12
4.3.2 Delkonklusion	13
4.4 Interessentanalyse af BookRight.....	14
4.4.1 Uddybning af interessenter	15
4.4.2 Interessenternes indflydelse og interesse	16
4.4.3 Vurdering af organisatoriske forandringer	17
4.4.4 Delkonklusion	18
4.5 Overvejelser om IT-Governance og bæredygtigt IT	18
4.5.1 Delkonklusion	18
4.6 Refleksion over innovative metoder.....	18
4.6.1 Double Diamond som refleksionsramme.....	19
4.6.2 Refleksion over anvendelsen	19

4.6.3 Delkonklusion	20
5. Kravspecifikation	20
5.1 Funktionelle krav og FURPS+	20
5.2 Ikke-funktionelle krav og FURPS+	23
5.3 MoSCoW-prioritering	25
5.4 Delkonklusion.....	27
6. Arkitektur og systemdesign.....	27
6.1 Clean Architecture og Dependency Rule	27
6.2 Domænemodel og Domain-Driven Design.....	27
6.3 CQS og Facade-lag	28
6.4 UML-modellering.....	29
6.4.1 Use case-diagram	29
6.4.2 Klassediagram	30
6.4.3 Sekvensdiagram for Command	31
6.4.4 Sekvensdiagram for Query	32
6.5 C4-modellen	33
6.5.1 Context-diagram – C1	33
6.5.2 Container-diagram – C2.....	34
6.6 Architecture Decision Records	34
6.6.1 ADR-001: Valg af Clean Architecture med Facade-lag	34
6.6.2 ADR-002: Valg af Command Query Separation	35
6.6.3 Delkonklusion	35
7. Implementering og Programmering I	36
7.1 Realisering af Clean Architecture i solutionen.....	36
7.2 Implementering af domænemodellen	37
7.3 Implementering af CQS og Facade	38
7.4 Blazor-brugergrenseflade og Dependency Injection	38
7.5 Designmønstre og SOLID	40
7.6 Rabatberegning med Strategy Pattern og parallel evaluering	41
7.7 EF Core Code First i implementeringen	43

7.8 Asynkrone databasekald og parallel udførelse	44
7.9 Delkonklusion.....	44
8. Teknologi I.....	45
8.1 Code First og normalisering.....	45
8.2 Proces, tråde og I/O	46
8.3 I/O-bound og CPU-bound parallelisme.....	46
8.4 Delkonklusion.....	47
9. Test og kvalitetssikring.....	47
9.1 Teststrategi	47
9.2 Unit tests for Domain-laget.....	47
9.3 Unit tests for Use Cases med Moq	48
9.4 Testresultater	49
10. Dokumentation af processen.....	50
10.1 Sprint 0 - Projektgrundlag og planlægning.....	50
10.2 Sprint 1 - Domænemodel og centrale forretningsregler	50
10.3 Sprint 2 - Use Cases, Facade og kvalitetssikring	51
10.4 Sprint 3 - Infrastruktur, rabatberegning og forbedringer	51
10.5 Sprint 4 - Brugergrænseflade i Blazor	52
10.6 Sprint 5 - Færdiggørelse, kvalitetssikring og rapportfærdiggørelse.....	52
10.7 Procesmæssige udfordringer og læring fra retrospectives	53
11. Perspektivering	54
12. Konklusion.....	55
13. Litteraturliste	57
14. Bilag	58

1. Indledning

1.1 Præsentation af BookRight Klinik & Wellness ApS

Digitalisering af administrative arbejdsgange spiller en væsentlig rolle for virksomheder, hvor planlægning, ressourceudnyttelse og kundehåndtering er centrale elementer i den daglige drift. Dette gælder særligt for klinikker og behandlingssteder, hvor bookinger skal koordineres mellem kunder, behandlere, behandlingstyper, lokationer og tilgængelige ressourcer.

BookRight Klinik & Wellness ApS er en fiktiv dansk virksomhed, der driver tre klinikker i Vejle-området. Virksomheden tilbyder behandlinger inden for fysioterapi, sportsmassage, akupunktur og kostvejledning og beskæftiger i alt 12 behandlere med forskellige specialer og autorisationer. BookRight betjener ca. 400 kunder månedligt og forventer at udvide med en fjerde klinik inden for det kommende år. Virksomhedens størrelse og forventede vækst medfører derfor et øget behov for en struktureret administration af bookinger, behandlere, klinikker og behandlingsrum.

1.2 Problemstilling

BookRights nuværende bookingproces er baseret på telefonisk booking, et fælles regneark og håndskrevne noter i receptionen. Denne manuelle arbejdsgang medfører risiko for dobbeltbookinger, tabte aftaler og manglende overblik over behandlernes belægning. Planlægningen kompliceres yderligere af, at behandlere kan arbejde på tværs af klinikker og har forskellige specialer og autorisationer, som afgør, hvilke behandlinger de kan udføre.

Den nuværende løsning understøtter derfor ikke i tilstrækkelig grad virksomhedens administrative behov eller forventede vækst. Der er behov for et internt administrationssystem, som kan samle booking- og kundehåndtering i én løsning, understøtte korrekt planlægning af behandlere og klinikker, reducere administrative fejl samt give et bedre datagrundlag for ledelsesmæssig opfølgning.

1.3 Projektets formål

Formålet med projektet er at analysere, designe og implementere en prototype af et internt administrationssystem til BookRight. Systemet målrettes receptionisternes arbejdsgange og skal understøtte oprettelse og administration af bookinger, registrering af kunder, håndtering af afbud og

registrering af kunders ankomst. Derudover skal løsningen bidrage til at forebygge dobbeltbookinger, understøtte korrekt pris- og rabatberegning, visualisere belægning og kalenderdata samt skabe grundlag for relevant rapportering til ledelsen.

Prototypen udvikles i C#/ .NET 10 med Blazor som brugergrænseflade og med udgangspunkt i Clean Architecture, Domain-Driven Design, Command Query Separation, SOLID-principper og automatiserede tests. Projektet har dermed både et teknisk og et forretningsmæssigt sigte, idet løsningen skal demonstrere en fagligt begrundet systemarkitektur og samtidig adressere BookRights konkrete administrative udfordringer.

2. Problemformulering og afgrænsning

2.1 Problemformulering

Hvordan kan der udvikles en prototype af et internt administrationssystem til BookRight Klinik & Wellness ApS, som gennem en struktureret og testbar softwarearkitektur understøtter receptionisternes bookingarbejde, reducerer administrative fejl og skaber bedre grundlag for planlægning, prisberegning og ledelsesinformation?

2.2 Underspørgsmål

- Hvordan kan BookRights nuværende arbejdsproces analyseres med henblik på at identificere centrale administrative udfordringer og forbedringsmuligheder?
- Hvilke funktionelle og ikke-funktionelle krav bør prioriteres for at understøtte receptionisternes arbejde og BookRights forretningsmæssige behov?
- Hvordan kan systemets domæne og arkitektur struktureres gennem Domain-Driven Design, Clean Architecture, CQS og et Facade-lag?
- Hvordan kan centrale funktioner som bookinghåndtering, validering af overlap og kapacitet samt pris- og rabatberegning implementeres i en prototype?
- Hvordan kan automatiserede tests og øvrig kvalitetssikring anvendes til at verificere systemets domæneregler og Use Cases?
- Hvordan kan systemet bidrage til organisatoriske og forretningsmæssige forbedringer for BookRight, herunder mere effektive arbejdsgange, færre fejl og bedre beslutningsgrundlag?

2.3 Afgrænsning

Projektet afgrænses til udviklingen af en prototype af et internt administrationssystem til BookRight, hvor systemets brugerrolle er receptionisten. Prototypen fokuserer på centrale arbejdsgange inden for booking, kundes håndtering, behandlere, klinikker, bookingstatus samt pris- og rabatberegning.

Løsningen omfatter ikke en selvbetjeningsløsning for kunder, online betaling eller en fuldt implementeret driftsløsning til produktion. Der arbejdes heller ikke med adgangsstyring mellem flere brugerroller, da casen definerer receptionisten som systemets eneste brugerrolle. Afgrænsningen er foretaget for at prioritere de funktioner, der er mest centrale for BookRights bookingproces, og som kan realiseres og dokumenteres inden for projektperiodens rammer.

3. Metode

3.1 Modifieret Scrum som systemudviklingsmetode

Projektet gennemføres med udgangspunkt i en modificeret anvendelse af Scrum som systemudviklingsmetode. Scrum er valgt, fordi projektet omfatter analyse, design, implementering, test og dokumentation inden for en afgrænset periode, hvor der løbende er behov for prioritering, koordinering og evaluering. Metoden er tilpasset vores arbejdsform og praktiske rammer med henblik på at skabe en hensigtsmæssig arbejdsfordeling og sikre fremdrift i projektet. Der anvendes således centrale Scrum-elementer som product backlog, sprints, Sprint Planning, Daily Stand-ups, Sprint Review og Sprint Retrospective, men i en form, der tager højde for vores individuelle behov og geografiske forhold.

Henrik varetager rollen som Scrum Master og har ansvar for koordinering, opgavefordeling og opfølgning på gruppens fremdrift. Jan, som underviser, fungerer i projektets ramme som Product Owner med fokus på kravafklaring og vurdering af, om løsningen udvikles i overensstemmelse med case beskrivelsen.

Arbejdet er struktureret gennem en product backlog, hvor kravene fra casen er nedbrudt i Epics, Features, User Stories og Tasks. Denne struktur har gjort det muligt at omsætte overordnede forretningsmæssige behov til konkrete og håndterbare udviklingsopgaver. Prioriteringen af arbejdet



har særligt været rettet mod de funktioner, der er centrale for BookRights bookingproces og den samlede prototype, herunder bookinghåndtering, forebyggelse af dobbeltbookinger samt pris- og rabatberegning.

Projektarbejdet er organiseret i seks sprints, benævnt Sprint 0 til Sprint 5. Sprint Planning er afholdt hver mandag ved fysisk fremmøde, hvor vi har gennemgået backloggen, udvalgt relevante user Stories og fordelt konkrete tasks for det kommende sprint. De daglige statusmøder er gennemført som korte Daily Stand-ups via Discord, da afstandsmæssige forhold har gjort fysisk fremmøde uhensigtsmæssigt på daglig basis. På møderne har vi orienteret om gennemført arbejde, planlagte aktiviteter og eventuelle udfordringer, der kunne påvirke sprintens fremdrift.

Hver fredag er der ved fysisk fremmøde afholdt Sprint Review og Sprint Retrospective. Ved Sprint Review har vi gennemgået den funktionalitet, der er udviklet i sprinten, f.eks. domæneregler, Use Cases, brugergrænseflade eller tests. Formålet har været at vurdere, om det udviklede resultat understøtter projektets krav og BookRights behov. Ved Sprint Retrospective har vi evalueret samarbejdet og arbejdsprocessen med fokus på velfungerende elementer, identificerede udfordringer og nødvendige justeringer til det efterfølgende sprint. Dette har gjort det muligt løbende at tilpasse den modificerede Scrum-proces til projektets udvikling.

Det konkrete sprintforløb, herunder sprintenes faglige fokus, centrale resultater samt erfaringer fra Reviews og Retrospectives, dokumenteres nærmere i afsnittet ”Dokumentation af processen”.

3.2 Samarbejde og projektorganisering

Som grundlag for samarbejdet har vi udarbejdet en gruppekontrakt, der fastlægger fælles forventninger til arbejdsindsats, kommunikation, deadlines, fremmøde samt håndtering af sygdom eller øvrige forhold, der kan påvirke deltagelsen i projektet. Gruppektrakten har haft til formål at skabe tydelige rammer for samarbejdet og reducere risikoen for uklarheder i forbindelse med opgavefordeling og ansvar.

Derudover har vi fastlagt fælles kodestandarder for at sikre ensartethed i kildekoden.

Kodestandarderne omfatter blandt andet principper for navngivning, strukturering af klasser og metoder samt læsbar og vedligeholdelsesvenlig kode. Dette understøtter samarbejdet, fordi kode



udviklet af forskellige gruppemedlemmer lettere kan forstås, gennemgås og videreudvikles af resten af gruppen.

Til versionsstyring anvendes GitHub med en fælles udviklingsbranch, DEV, hvor vores implementeringsarbejde samles under projektforsløbet. Strategien er valgt som en tilpasning til vores arbejdsform og erfaringer med versionsstyring, idet den skaber ét fælles udviklingsgrundlag for den igangværende funktionalitet. Samtidig stiller arbejdet i en fælles branch krav om tydelige commits, løbende koordinering, Code Review og overholdelse af vores kodenstandards. Commits anvendes til at dokumentere udviklingen af konkrete funktioner og bidrager dermed til sporbarhed mellem krav, implementering og test.

3.3 Analyse-, krav- og dokumentationsmetoder

Som en del af forretningsanalysen anvendes en As-Is-analyse til at undersøge BookRights nuværende arbejdsproces og identificere centrale udfordringer ved den manuelle bookinghåndtering. På baggrund heraf anvendes en To-Is-analyse til at beskrive, hvordan et digitalt administrationssystem kan understøtte en mere sammenhængende og effektiv fremtidig arbejdsproces. Analyserne danner samlet grundlag for systemets krav og vurderingen af de organisatoriske forandringer, som digitaliseringen medfører.

Derudover anvendes en SWOT-analyse til at vurdere BookRights interne styrker og svagheder samt eksterne muligheder og trusler i relation til digitaliseringen. Analysen anvendes blandt andet til at belyse, hvordan systemet kan reducere administrative fejl, skabe bedre overblik og understøtte virksomhedens fremtidige vækst. Desuden udarbejdes en interessentanalyse, hvor centrale aktører og deres behov identificeres og vurderes. Receptionisterne betragtes som systemets primære brugere, mens ledelsen, behandlerne, kunderne og administrationen påvirkes af løsningen på forskellige måder.

Kravarbejdet struktureres i funktionelle og ikke-funktionelle krav. De funktionelle krav beskriver, hvilke handlinger systemet skal kunne understøtte, f.eks. bookinghåndtering, kunderegistrering, validering af overlap og kapacitet samt pris- og rabatberegning. De ikke-funktionelle krav omhandler løsningens kvalitetsmæssige egenskaber, herunder brugervenlighed, testbarhed, vedligeholdelsesvenlighed og skalerbarhed.

Til prioritering af kravene anvendes MoSCoW, hvor kravene kategoriseres som Must have, Should have, Could have og Won't have. Metoden anvendes til at sikre, at udviklingsarbejdet fokuserer på de funktioner, der er mest centrale for BookRights kerneproces og den demonstrerbare prototype. Den konkrete prioritering fremgår af afsnittet "Kravspecifikation".

De funktionelle og ikke-funktionelle krav kategoriseres desuden med udgangspunkt i FURPS+. Modellen anvendes til systematisk at strukturere krav inden for Functionality, Usability, Reliability, Performance og Supportability, mens plustegnet omfatter teknologiske og arkitektoniske bindinger, f.eks. C#/.NET, Blazor, EF Core Code First og Clean Architecture. Den konkrete anvendelse af FURPS+ fremgår ligeledes af "kravspecifikationen".

Til modellering af systemet anvendes UML, herunder Use Case-diagram, klassediagram samt sekvensdiagrammer for henholdsvis en Command og en Query. Systemets overordnede arkitektur dokumenteres gennem C4-modellen med et Context-diagram og et Container-diagram. Derudover anvendes Architecture Decision Records (ADR) til at dokumentere centrale arkitektoniske beslutninger med fokus på beslutningernes kontekst, valg og konsekvenser. Diagrammer og ADR'er præsenteres og analyseres nærmere i afsnittet "Arkitektur og systemdesign".

3.4 Digitale hjælpeværktøjer og generativ AI

I projektarbejdet anvendes ChatGPT og GitHub Copilot som understøttende værktøjer og ikke som erstatning for vores egne faglige vurderinger og beslutninger. ChatGPT er blandt andet anvendt til idéudvikling i forbindelse med systemets farvetema, udarbejdelse af baggrundsbillede til rapportens forside og design af BookRight-logo. Derudover er værktøjet anvendt som støtte ved generering af syntetiske testdata samt som faglig sparringspartner i forbindelse med forståelsen af tekniske emner, f.eks. LINQ.

GitHub Copilot er anvendt som støtte i den løbende udviklingsproces, herunder opsummeringer af implementeret funktionalitet samt formulering af commit-beskeder. Dette har bidraget til en mere ensartet dokumentation af udviklingsarbejdet i GitHub.

4. Forretningsanalyse

4.1 As-Is-analyse: Nuværende arbejdsproces

BookRights nuværende booking- og administrationsproces er i høj grad manuel. Kundernes bookinger modtages telefonisk og registreres gennem en kombination af et fælles regneark og håndskrevne notater i receptionen. Receptionisterne skal derfor manuelt koordinere oplysninger om kunder, behandlinger, behandlere, klinikker og tidspunkter. Processen kompliceres yderligere af, at behandlere kan arbejde på tværs af flere klinikker og samtidig har forskellige specialer og autorisationer, som afgør, hvilke behandlinger de kan udføre.

Den nuværende arbejdsproces er sårbar, fordi relevante oplysninger håndteres i flere forskellige informationskilder uden automatisk validering eller et samlet overblik. Dette medfører risiko for dobbeltbookinger, tabte aftaler og uhensigtsmæssig planlægning af behandlernes tid og klinikkernes kapacitet. Samtidig begrænser den manuelle registrering virksomhedens mulighed for løbende at følge belægning, bookinghistorik og økonomiske resultater på et ensartet og pålideligt datagrundlag.

Den manuelle proces har desuden en direkte forretningsmæssig konsekvens. Ifølge casen anvendes der anslået 15 timer ugentligt på administration, mens fejl og manglende opfølgning har medført et estimeret tab på 8–10 % af potentielle bookinger. As-Is-analysen viser dermed, at den eksisterende arbejdsgang ikke i tilstrækkelig grad understøtter BookRights nuværende drift eller virksomhedens forventede vækst med en kommende fjerde klinik. Der er derfor et tydeligt behov for en mere sammenhængende og systemunderstøttet administrationsproces.

De identificerede udfordringer danner grundlag for projektets digitale løsning, hvor samlet bookinghåndtering, automatiske valideringer, prisberegning og strukturerede data skal bidrage til færre fejl, bedre overblik og mere effektiv administration.

4.2 To-Be-analyse: Fremtidig arbejdsproces med BookRight-systemet

I den fremtidige arbejdsproces skal BookRight-systemet fungere som et samlet internt administrationssystem for receptionisterne. I stedet for at håndtere bookinger gennem telefon, regneark og håndskrevne noter skal receptionisten kunne registrere kunder, oprette og administrere bookinger samt håndtere afbud og kunders ankomst i én digital løsning. Systemet skal dermed samle de centrale oplysninger om kunder, behandlinger, behandlere, klinikker og tidsrum, så arbejdsprocessen bliver mere ensartet og overskuelig.

Ved oprettelse af en booking skal systemet understøtte receptionisten gennem automatiske kontroller. Det indebærer blandt andet, at systemet skal kunne kontrollere, om den valgte behandler kan udføre behandlingen, om behandleren og klinikken er ledige i det ønskede tidsrum, samt om klinikkens kapacitet er tilstrækkelig. Herved reduceres risikoen for dobbeltbookinger og fejl i planlægningen. Systemet skal desuden kunne beregne den mest fordelagtige pris for kunden på baggrund af relevante rabatmuligheder, f.eks. loyalitetsniveau, kampagner og fødselsdagsmåned. Prisberegningen bliver dermed mere konsekvent og mindre afhængig af manuel vurdering.

Den fremtidige proces skal samtidig skabe bedre overblik for både receptionister og ledelse. Receptionisten skal kunne se og administrere bookinger samt deres status i systemet, mens et visuelt dashboard skal synliggøre behandlernes kalendere og klinikkernes belægning. Derudover skal systemet kunne danne omsætningsrapporter, som kan understøtte ledelsens opfølgning og beslutningstagning. Bookinghistorik og strukturerede data vil således kunne anvendes som grundlag for både prisberegning, statistik og rapportering.

To-Be-analysen viser, at BookRight-systemet ikke alene digitaliserer den eksisterende bookingproces, men også forbedrer den gennem validering, automatisering og et samlet datagrundlag. Den fremtidige arbejdsproces forventes derfor at reducere administrative fejl, mindske tidsforbruget ved manuel koordinering og gøre virksomheden bedre rustet til øget aktivitet og en mulig udvidelse med en fjerde klinik. Systemets værdi afhænger dog af, at løsningen er brugervenlig og integreres som det centrale arbejdsredskab i receptionisternes daglige administration.

4.3 SWOT-analyse

En SWOT-analyse anvendes til at identificere virksomhedens interne styrker og svagheder samt de eksterne muligheder og trusler, der kan påvirke projektets forretningsmæssige relevans. I BookRights tilfælde er analysen særligt relevant, fordi virksomheden står over for en digital transformation fra en manuel bookingproces til et samlet internt administrationssystem.



4.3.1 Uddybning af SWOT-analysen

BookRights væsentligste styrke er en etableret forretning med flere klinikker, et bredt behandlingstilbud og et betydeligt kundegrundlag. Behandlernes forskellige specialer og autorisationer giver virksomheden faglig bredde og mulighed for at imødekomme forskellige kundebehov. Dette udgør et solidt grundlag for et digitalt system, der kan understøtte og effektivisere den eksisterende drift.



Virksomhedens væsentligste svaghed er den nuværende manuelle bookingproces, som bygger på telefonisk booking, regneark og håndskrevne noter. Arbejdsgangen øger risikoen for dobbeltbookinger, tabte aftaler og manglende overblik over behandlere og klinikkapacitet. Komplexiteten forstærkes af, at behandlere arbejder på tværs af klinikker og har forskellige kompetencer, hvilket gør manuel planlægning fejlbarlig.

Digitalisering udgør en central mulighed for BookRight. Et samlet administrationssystem kan understøtte booking, kundehåndtering, kapacitetsstyring, prisberegning og rapportering i én løsning. Herved kan receptionisternes arbejde effektiviseres, mens ledelsen opnår et bedre datagrundlag for opfølgning på belægning og drift. Automatiseret rabatberegning kan samtidig bidrage til en mere ensartet og korrekt prisfastsættelse.

Den væsentligste trussel er, at de eksisterende administrative udfordringer forstærkes i takt med virksomhedens vækst. Hvis BookRight udvider uden at forbedre sine arbejdsgange, kan fejl og tabte bookinger øges. Samtidig afhænger værdien af et nyt system af, at det er brugervenligt og anvendes konsekvent. Endelig stiller digital registrering af kunde- og helbredsoplysninger krav til ansvarlig og sikker databehandling.

4.3.2 Delkonklusion

SWOT-analysen viser, at BookRight har et solidt forretningsmæssigt grundlag, men samtidig er begrænset af en manuel og fejlbarlig bookingproces. Et digitalt administrationssystem kan derfor skabe væsentlig værdi gennem mere effektiv planlægning, færre administrative fejl og bedre grundlag for virksomhedens videre vækst.

4.4 Interessentanalyse af BookRight

Interessentanalysen identificerer de aktører, der påvirker eller påvirkes af BookRight-systemet.

Analysen er relevant, fordi overgangen fra manuelle arbejdsgange til et digitalt administrationssystem ændrer både receptionisternes arbejde og grundlaget for planlægning, kundeservice og ledelsesmæssigt overblik.

Centrale interessenter:

Interessentanalyse – BookRight

Interessent	Rolle i forhold til systemet	Interesse/behov	Indflydelse
Receptionister	Primære brugere af systemet	Skal kunne oprette og administrere bookinger, registrere kunder, håndtere afbud og betjene kunder ved ankomst.	Høj
BookRights ledelse	Beslutningstagere og modtagere af rapportering	Ønsker bedre overblik over drift, omsætning, belægning og kapacitetsudnyttelse.	Høj
Behandlere	Ressourcer i bookingsystemet	Har behov for korrekt kalenderstyring, realistisk planlægning og undgåelse af dobbeltbookinger.	Mellem
Kunder	Modtagere af behandlinger og indirekte brugere af systemet	Forventer korrekt booking, god service, fair prisberegning og mulighed for opfølgning.	Mellem
Administration/økonomi	Bruger data til rapportering og opfølgning	Har behov for pålidelige data om omsætning, rabatter, bookinger og kundehistorik.	Mellem
Projektgruppen/udviklingsteamet	Udvikler prototypen	Skal omsætte kravene til en teknisk løsning med Clean Architecture, Blazor, tests og dokumentation.	Høj i projektperioden
Klinikkerne som organisatoriske enheder	Fysiske lokationer med begrænset kapacitet	Har behov for styring af behandlingsrum, åbningstider og belægning.	Mellem
Myndigheder/lovgivning	Ekstern ramme for datasikkerhed	Har indirekte betydning, da systemet registrerer kundeoplysninger og helbredsnotater.	Lav direkte, høj indirekte

4.4.1 Uddybning af interessenter

Receptionisterne er systemets primære brugere og har derfor høj interesse i en brugervenlig og stabil løsning. Systemet skal understøtte deres arbejde med booking, kunderegistrering, afbud og kunders ankomst samt reducere behovet for manuel kontrol.

Ledelsen har interesse i, at systemet effektiviserer driften og skaber et bedre grundlag for opfølgning på belægning, omsætning og kapacitet. Ledelsen har dermed høj indflydelse på vurderingen af løsningens forretningsmæssige værdi.

Behandlerne er indirekte berørt, fordi systemet skal sikre korrekt planlægning i forhold til klinik, tidspunkt og autorisation. En præcis bookinglogik kan dermed reducere overlap og fejlagtige behandlingstilknytninger.

Kunderne påvirkes indirekte gennem bookingprocessens kvalitet. For dem er korrekt registrering, færre fejl og ensartet prisberegning væsentlige forhold for en professionel og tillidsvækkende serviceoplevelse.

Administration og økonomi har behov for pålidelige data om bookinger, kundehistorik, rabatter og omsætning. Et digitalt system kan derfor reducere manuelt arbejde og styrke grundlaget for rapportering og opfølgning.

Vi som projektgruppe har høj indflydelse i udviklingsperioden, da vi omsætter BookRights behov og case krav til en teknisk prototype. De valgte prioriteringer og arkitekturvalg har dermed direkte betydning for løsningens kvalitet og anvendelighed.

Klinikkerne udgør organisatoriske enheder med begrænset kapacitet i form af behandlingsrum og åbningstider. Systemet skal derfor understøtte korrekt kapacitetsstyring og forebygge overbooking.

Myndigheder og lovgivning har lav direkte involvering, men høj indirekte betydning, da systemet håndterer kundeoplysninger og eventuelle helbredsnotater. Dette stiller krav til ansvarlig databehandling og informationssikkerhed.

4.4.2 Interessenternes indflydelse og interesse

Interessentanalyse – BookRight

Kategori	Interessenter	Håndtering
Høj indflydelse / høj interesse	Receptionister, ledelse, projektgruppen	Skal inddrages tæt i krav, prioriteringer, test og evaluering.
Lavere indflydelse / høj interesse	Kunder, behandlere, administration	Skal analyseres grundigt, da systemets værdi i høj grad afhænger af deres behov.
Mellem indflydelse / mellem interesse	Klinikkerne som organisatoriske enheder	Skal indgå i krav til kapacitet, åbningstider og belægningsstyring.
Lav direkte indflydelse / høj indirekte betydning	Myndigheder/lovgivning	Skal håndteres gennem krav til datasikkerhed, ansvarlig databehandling og adgangskontrol.

4.4.3 Vurdering af organisatoriske forandringer

Organisatoriske forandringer – BookRight

Interessent/område	Organisatorisk forandring
Receptionister	Går fra manuel registrering og kontrol i flere informationskilder til digital booking, systemvalidering og samlet kundehåndtering.
Ledelsen	Får adgang til mere strukturerede data om belægning, omsætning og kapacitetsudnyttelse som grundlag for opfølgning og beslutninger.
Behandlere	Bliver planlagt mere systematisk på tværs af klinikker, tidspunkter, behandlingstyper og autorisationer.
Kunder	Forventes at opleve mere præcis booking, korrekt prisberegning og færre fejl i den administrative proces.
Administration og databehandling	Får et samlet datagrundlag til rapportering og opfølgning, men også et større ansvar for korrekt og sikker håndtering af oplysninger.

Implementeringen af BookRight-systemet ændrer virksomhedens bookingproces fra manuel koordinering til digital og systemunderstøttet administration. Hvor oplysninger tidligere blev håndteret gennem telefonbooking, regneark og håndskrevne noter, skal systemet samle booking- og kundeoplysninger samt understøtte validering af tid, behandler, klinikkapacitet og pris.

Receptionisterne påvirkes mest direkte, da systemet ændrer deres daglige arbejdsgange og reducerer behovet for manuel kontrol. Overgangen forudsætter samtidig, at systemet er brugervenligt og anvendes konsekvent. Ledelsen kan opnå et mere struktureret datagrundlag for opfølgning på drift, belægning og kapacitet, mens behandlere og kunder indirekte får fordel af mere præcis planlægning, færre fejl og mere ensartet prisberegning.

Digitaliseringen medfører desuden et øget ansvar for datakvalitet og sikker håndtering af kundeoplysninger og eventuelle helbredsnotater. Systemet udgør derfor ikke alene en teknisk

løsning, men også en organisatorisk ændring, der kræver tydelige arbejdsgange og ansvarlig anvendelse.

4.4.4 Delkonklusion

Analysen viser, at BookRight-systemet ændrer bookingprocessen fra manuel koordinering til en mere ensartet og datadrevet administration. Receptionisterne påvirkes mest direkte, mens ledelse, behandlere og kunder forventes at opnå bedre overblik, mere præcis planlægning og færre administrative fejl. Gevinsterne forudsætter dog en brugervenlig løsning samt ansvarlig håndtering af kunde- og helbredsoplysninger.

4.5 Overvejelser om IT-Governance og bæredygtigt IT

BookRight-systemet skal understøtte virksomhedens daglige drift og håndtere kundeoplysninger samt eventuelle helbredsnotater. IT-Governance er derfor relevant i forhold til ansvarlig databehandling, adgangsbegrænsning, datakvalitet og dokumentation. I projektet understøttes dette gennem Clean Architecture, automatiserede tests og versionsstyring, som bidrager til sporbarhed og vedligeholdelsesvenlighed.

Bæredygtigt IT relaterer sig i projektet især til ressourceeffektiv administration og teknisk enkelhed. Et digitalt system kan reducere manuelle arbejdsgange og forbedre udnyttelsen af klinikkernes og behandlernes kapacitet. Teknisk bør løsningen samtidig begrænse unødvendig datalagring og overflødige databasekald, så systemet forbliver effektivt og skalerbart.

4.5.1 Delkonklusion

Samlet set bidrager overvejelser om IT-Governance og bæredygtigt IT til, at BookRight-systemet udvikles som en ansvarlig, dokumenterbar og ressourcebevidst løsning.

4.6 Refleksion over innovative metoder

Innovation forstås i projektet som anvendelsen af IT til at forbedre BookRights eksisterende bookingproces. Prototypen har derfor fokus på at reducere administrative fejl og skabe bedre overblik gennem samlet bookinghåndtering, validering af overlap samt automatiseret prisberegning.

4.6.1 Double Diamond som refleksionsramme

Double Diamond anvendes som en supplerende refleksionsramme med faserne Discover, Define, Develop og Deliver. Modellen synliggør sammenhængen mellem de identificerede problemer og den udviklede løsning, mens den modificerede Scrum-proces strukturerer det iterative udviklingsarbejde.

Double Diamond – BookRight

Fase	Anvendelse i BookRight-projektet	Betydning for løsningen
Discover – undersøge	BookRights nuværende arbejdsproces analyseres gennem As-Is-analyse, SWOT-analyse og interessentanalyse. Her identificeres blandt andet dobbeltbookinger, tabte aftaler, manglende overblik og tidskrævende administration.	Fasen skaber forståelse for de konkrete problemer, som systemet skal reducere eller løse.
Define – afgrænse	På baggrund af analysen konkretiseres problemstillingen og systemets centrale krav. Løsningen afgrænses til et internt administrationssystem for receptionister med fokus på booking, kapacitetsstyring, prisberegning og rapportering.	Fasen sikrer, at projektet prioriterer de funktioner, der har størst forretningsmæssig betydning for BookRight.
Develop – udvikle	Projektgruppen designer og implementerer en prototype med blandt andet bookingflow, kontrol af overlap, validering af behandler og klinikkapacitet, booking-statusser samt understøttelse af pris- og rabatberegning.	Fasen omsætter de identificerede behov til konkrete tekniske funktioner og forretningsregler.
Deliver – levere og vurdere	Løsningen afprøves gennem automatiserede tests og vurderes i forhold til de opstillede krav og de identificerede arbejdsgange. Prototypen skal demonstrere, hvordan systemet kan understøtte receptionisternes arbejde og BookRights drift.	Fasen giver grundlag for at vurdere, om løsningen er funktionelt korrekt og relevant for virksomheden.

4.6.2 Refleksion over anvendelsen

Double Diamond skaber sammenhæng mellem BookRights identificerede udfordringer og den udviklede løsning. Systemets funktioner udspringer således af konkrete problemer i den nuværende arbejdsgang, f.eks. at overlap- og kapacitetsvalidering skal reducere risikoen for fejlbookinger.

Modellen supplerer den modificerede Scrum-proces ved at fastholde fokus på problemforståelse og forretningsmæssig værdi, mens Scrum strukturerer den iterative udvikling. Anvendelsen er dog begrænset af, at projektet baseres på en case og ikke på direkte brugerafprøvning. En videreudvikling bør derfor inddrage receptionister gennem f.eks. interviews eller brugertests.

4.6.3 Delkonklusion

Double Diamond har bidraget til at koble BookRights udfordringer med projektets prioriteringer og løsning. Modellen understøtter dermed, at prototypen ikke blot digitaliserer den eksisterende proces, men søger at forbedre den gennem validering og mere struktureret administration.

5. Kravspecifikation

5.1 Funktionelle krav og FURPS+

F1.Booking

ID	KRAV – "Systemet skal kunne..."	FURPS+
F1.1	Oprette bookinger.	FUNCTIONALITY
F1.2	Ændre bookinger.	FUNCTIONALITY
F1.3	Søge blandt bookinger for at finde bestemt booking.	FUNCTIONALITY / USABILITY
F1.4	Understøtte bookinger på tværs af klinikker og behandlinger.	FUNCTIONALITY
F1.5	Sikre, at der ikke kan opstå dobbeltbookinger.	FUNCTIONALITY / RELIABILITY
F1.6	Markere bookinger med status som NoShow, Aflyst og Afsluttet.	FUNCTIONALITY
F1.7	Håndtere afbud fra kunder.	FUNCTIONALITY
F1.8	Håndtere check-in af kunder ved ankomst.	FUNCTIONALITY
F1.9	Understøtte kombinationsbookinger.	FUNCTIONALITY
F1.10	Validere, at en behandler er tilknyttet den valgte klinik.	FUNCTIONALITY / RELIABILITY
F1.11	Tage højde for klinikkens antal behandlingsrum.	FUNCTIONALITY / RELIABILITY
F1.12	Holde historik over alle bookinger for en given kunde, for at kunne fastsætte loyalitetsniveau til prisberegning.	FUNCTIONALITY

F.2 Kundeadministration

ID	KRAV – ”Systemet skal kunne...”	FURPS+
F2.1	Registrere kunder med: Fornavn, efternavn, telefon, e-mail, fødselsdato, adresse	FUNCTIONALITY
F2.2	Redigere kundeoplysninger.	FUNCTIONALITY
F2.3	Registrere evt. helbredsnotater om kunden.	FUNCTIONALITY
F2.4	Registrere evt. foretrukken behandler for kunden.	FUNCTIONALITY
F2.5	Søge blandt kunder for at finde bestemt kunde.	FUNCTIONALITY / USABILITY

F.3 Behandleradministration

ID	KRAV – ”Systemet skal kunne...”	FURPS+
F3.1	Registrere behandlere med: Navn, kontaktoplysninger, autorisationstype, autorisationsnummer og tilknyttede klinikker.	FUNCTIONALITY
F3.2	Sikre at behandlere kun kan udføre behandlinger, de er autoriserede til.	FUNCTIONALITY / RELIABILITY
F3.3	Vise et dashboard med behandlernes kalender.	FUNCTIONALITY / USABILITY

F.4. Klinikadministration

ID	KRAV – ”Systemet skal kunne...”	FURPS+
F4.1	Registrere klinikker med: Adresse, åbningstider og antal behandlingsrum.	FUNCTIONALITY
F4.2	Håndtere begrænsninger på antal behandlingsrum.	FUNCTIONALITY / RELIABILITY
F4.3	Vise et dashboard med klinikkernes belægning.	FUNCTIONALITY / USABILITY

F5. Pris- og rabatberegning

ID	KRAV – ”Systemet skal kunne...”	FURPS+
F5.1	Automatisk beregne prisen for en booking.	FUNCTIONALITY / RELIABILITY
F5.2	Understøtte forskellige rabattyper, såsom loyalitetsrabat, fødselsdagsrabat og kampagnerabat.	FUNCTIONALITY
F5.3	Automatisk vælge den rabat der giver kunden den laveste pris.	FUNCTIONALITY / RELIABILITY
F5.4	Sikre at rabatter ikke kan kombineres.	FUNCTIONALITY / RELIABILITY
F5.5	Vise prisen med og uden rabat, samt anvendt rabattype.	FUNCTIONALITY / USABILITY
F5.6	Systemet skal kunne håndtere et 15% tillæg for aften/weekend.	FUNCTIONALITY / RELIABILITY

F6. Rapportering

ID	KRAV – ”Systemet skal kunne...”	FURPS+
F6.1	Generere omsætningsrapporter.	FUNCTIONALITY
F6.2	Omsætningsrapporterne skal kunne eksporteres (f.eks. CSV-fil), for deling med ledelsen.	FUNCTIONALITY

5.2 Ikke-funktionelle krav og FURPS+

IF1. Brugervenlighed

ID	KRAV	FURPS+
IF1.1	Systemet skal være intuitivt og let at anvende for receptionister.	USABILITY
IF1.2	En receptionist skal kunne oprette en booking på maks. 2 minutter.	USABILITY

IF2. Pålidelighed og Tilgængelighed

ID	KRAV	FURPS+
IF2.1	Systemet skal være tilgængeligt i klinikkernes åbningstid.	RELIABILITY
IF2.2	Systemet skal have en opetid på minimum 99%.	RELIABILITY

IF3. Ydeevne

ID	KRAV	FURPS+
IF3.1	Systemet skal understøtte håndtering af min. 400 kunder pr. måned uden performanceproblemer.	PERFORMANCE / RELIABILITY
IF3.2	Blazor-brugergrænsefladen skal opleves gnidningsfrit.	PERFORMANCE / USABILITY
IF3.3	Operationer, såsom Søg og Gem, skal opleves gnidningsfrit.	PERFORMANCE / RELIABILITY

IF4. Vedligeholdelse

ID	KRAV	FURPS+
IF4.1	Systemet skal anvende DDD og Clean Architecture for nem vedligeholdelse og udvidelse.	SUPPORTABILITY / OTHER+
IF4.2	Systemet skal anvende SOLID-principperne.	SUPPORTABILITY / OTHER+
IF4.3	Systemet skal understøtte automatiserede tests	SUPPORTABILITY

IF5. Skalérbarhed

ID	KRAV	FURPS+
IF5.1	Systemets ressourcer skal understøtte oprettelse af nye klinikker med tilknyttede behandlere og øget antal kunder.	PERFORMANCE / SUPPORTABILITY

IF6. Dataintegritet

ID	KRAV	FURPS+
IF6.1	Systemet skal sikre korrekt relation mellem kunder, bookinger og behandlere.	RELIABILITY

IF7. Retligt

ID	KRAV	FURPS+
IF7.1	GDPR.	OTHER+

IF8. Teknologiske krav

ID	KRAV	FURPS+
IF8.1	Systemet skal designes og udvikles i C# .NET 10	OTHER+
IF8.2	Systemet skal anvende Entity Framework Core (Code First).	OTHER+
IF8.3	Systemets brugergrænseflade skal designes og implementeres med Blazor.	OTHER+
IF8.4	Systemet skal implementere et Facade-lag m. Interfaces & DTO'er som eneste kontrakt mod Blazor.	OTHER+
IF8.5	Dependency Injection skal anvendes.	OTHER+
IF8.6	Repository-pattern skal anvendes.	OTHER+
IF8.7	Strategy-pattern skal anvendes ved beregning af rabat.	OTHER+
IF8.8	Dataoverførsel skal være krypteret (HTTPS).	OTHER+
IF8.9	Command Query Separation (CQS) skal anvendes.	OTHER+

5.3 MoSCoW-prioritering

Must Have:

F1.1 - Oprette bookinger.

F1.4 - Understøtte bookninger på tværs af klinikker og behandlinger.

F1.5 - Sikre, at der ikke kan opstå dobbeltbookinger

F1.11 - Tage højde for klinikkens antal behandlingsrum.

F2.1 - Registrere kunder med: Fornavn, efternavn, telefon, e-mail, fødselsdato, adresse

F2.2 - Redigere kundeoplysninger.

F2.5 - Søg blandt kunder for at finde bestemt kunde.

F3.2 - Sikre at behandlere kun kan udføre behandlinger, de er autoriserede til.

F5.1 - Automatisk beregner prisen for en booking.

F5.3 - Automatisk vælge den rabat der giver kunden den laveste pris.

F5.4 - Sikre at rabatter ikke kan kombineres.

F5.5 - Vise prisen med og uden rabat, samt anvendt rabattype.



Should have:

F1.3 - Søge blandt bookinger for at finde bestemt booking.

F1.10 - Validere, at en behandler er tilknyttet den valgte klinik.

F1.12 - Holde historik over alle bookinger for en given kunde, for at kunne fastsætte loyalitetsniveau til prisberegning.

F2.3 - Registrere evt. helbredsnotater om kunden.

F3.1 - Registrere behandlere med: Navn, kontaktoplysninger, autorisationstype, autorisationsnummer og tilknyttede klinikker.

Could have:

F1.2 - Ændre bookinger.

F1.6 - Markere bookinger med status som NoShow, Aflys og Afsluttet.

F1.7 - Håndtere afbud fra kunder.

F1.8 - Håndtere check-in af kunder ved ankomst.

F1.9 - Understøtte kombinationsbookinger.

F2.4 - Registrere evt. foretrukken behandler for kunden.

F3.3 - Vise et dashboard med behandlernes kalender.

F4.1 - Registrere klinikker med: Adresse, åbningstider og antal behandlingsrum.

F4.2 - Håndtere begrænsninger på antal behandlingsrum.

F4.3 - Vise et dashboard med klinikkernes belægning.

F5.2 - Understøtte forskellige rabattyper, såsom loyalitetsrabat, fødselsdagsrabat og kampagnerabat.

F5.6 - Systemet skal kunne håndtere et 15% tillæg for aften/weekend.

F6.1 - Generere omsætningsrapporter.

F6.2 - Omsætningsrapporterne skal kunne eksporteres (f.eks. CSV-fil), for deling med ledelsen.

Won't have:

Vi har som udgangspunkt ikke haft nogen won't have.

5.4 Delkonklusion

Kravspecifikationen viser, at prototypens primære fokus er de funktioner, der direkte understøtter BookRights bookingproces. Bookingoprettelse, kontrol af overlap, validering af behandler og klinikkapacitet samt korrekt pris- og rabatberegning prioriteres højest, fordi disse funktioner adresserer de mest centrale problemer i den nuværende manuelle arbejdsgang. Funktioner som dashboard, rapportering og udvidet administration vurderes fortsat relevante, men prioriteres lavere, da de ikke er afgørende for at demonstrere løsningens centrale booking flow inden for projektets rammer.

6. Arkitektur og systemdesign

6.1 Clean Architecture og Dependency Rule

Løsningen er struktureret efter Clean Architecture, hvor systemets ansvar er opdelt i separate projekter for henholdsvis Domain, Use Case/Application, Facade, Infrastructure og UI. Domain-laget indeholder de centrale forretningsregler og er uafhængigt af tekniske detaljer. Use Case-laget koordinerer systemets handlinger, mens Facade-laget udgør kontrakten mellem brugergrænsefladen og den bagvedliggende applikationslogik. Infrastructure-laget håndterer persistens og datatilgang, og UI-laget præsenterer funktionaliteten for receptionisten gennem Blazor.

Denne lagdeling understøtter Dependency Rule, idet de indre lag ikke afhænger af de ydre tekniske lag. Facade-laget medfører dog en afvigelse fra princippet, idet Application-laget er afhængigt af Facade-laget. Dermed kan forretningslogikken udvikles og testes uafhængigt af f.eks. database og brugergrænseflade.

6.2 Domænemodel og Domain-Driven Design

Domain-laget er udarbejdet med udgangspunkt i principper fra Domain-Driven Design. I løsningen fungerer Booking som central Aggregate Root, da klassen håndterer bookingens tilstand og centrale forretningsregler, f.eks. statusændringer, ændring af tidspunkt og håndtering af deltagere.

Domænebegreber som Customer, Therapist, Clinic og Treatment repræsenteres som Entities, mens TimeSlot anvendes som Value Object til at repræsentere og validere et bookings tidsinterval.



Anvendelsen af TimeSlot samler regler om gyldige tidsintervaller og overlap ét sted i domænet. Dette bidrager til en mere konsistent domænemodel, der afspejler BookRights centrale forretningsregler. De centrale domæneobjekter og deres relationer fremgår af klassediagrammet under afsnit 6.4.2 Klassediagram.

6.3 CQS og Facade-lag

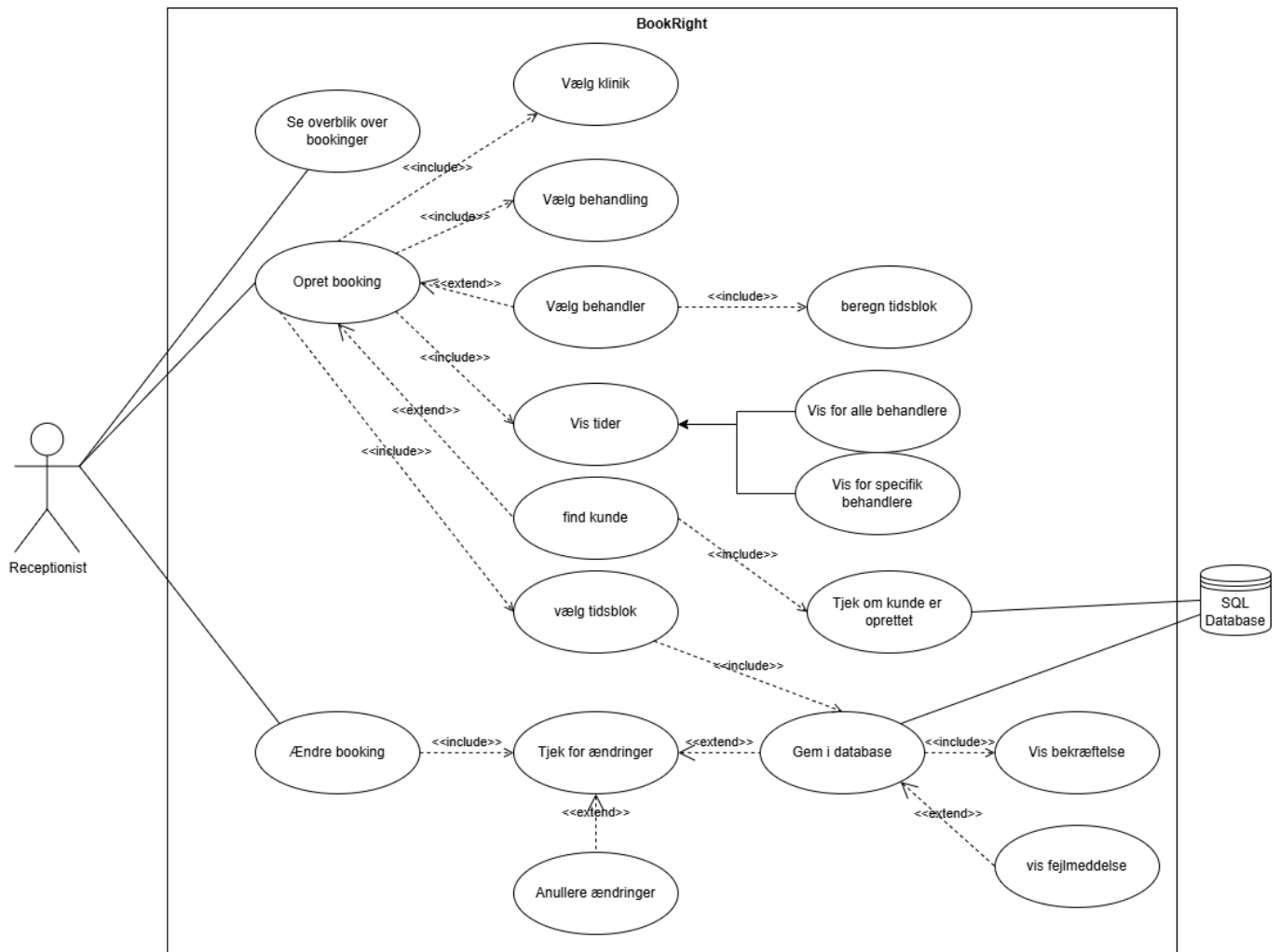
BookRight-systemet anvender Command Query Separation (CQS) ved at adskille handlinger, der ændrer systemets tilstand, fra forespørgsler, der alene henter data. Commands anvendes f.eks. ved oprettelse eller ændring af en booking, mens Queries anvendes til at hente data om bookinger, kunder, klinikker og behandlinger til brugergrænsefladen.

Facade-laget fungerer som den eneste kontrakt mellem Blazor-brugergrænsefladen og de bagvedliggende lag gennem interfaces og DTO'er. Blazor-komponenterne får dermed ikke direkte adgang til Domain- eller Infrastructure-laget, hvilket reducerer koblingen og understøtter en mere vedligeholdelsesvenlig løsning.

Den konkrete anvendelse af CQS og Facade-laget fremgår af sekvensdiagrammerne for henholdsvis en Command og en Query i afsnit 6.4.3 og 6.4.4.

6.4 UML-modellering

6.4.1 Use case-diagram



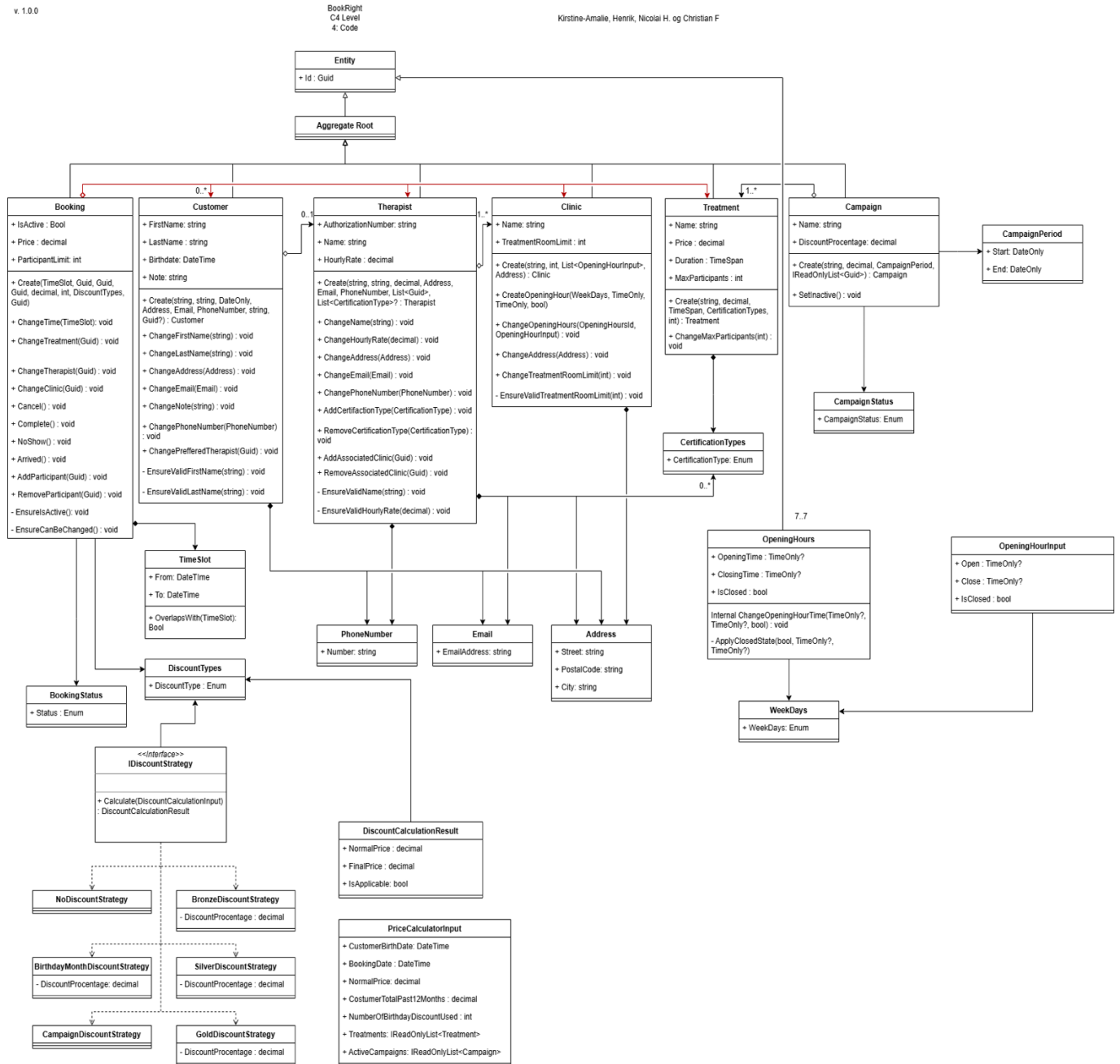
Use case-diagrammet viser systemets centrale funktioner set fra receptionistens perspektiv. Receptionisten er systemets direkte bruger og varetager de administrative opgaver med kunder, bookinger og bookingstatus.

6.4.2 Klassediagram

v. 1.0.0

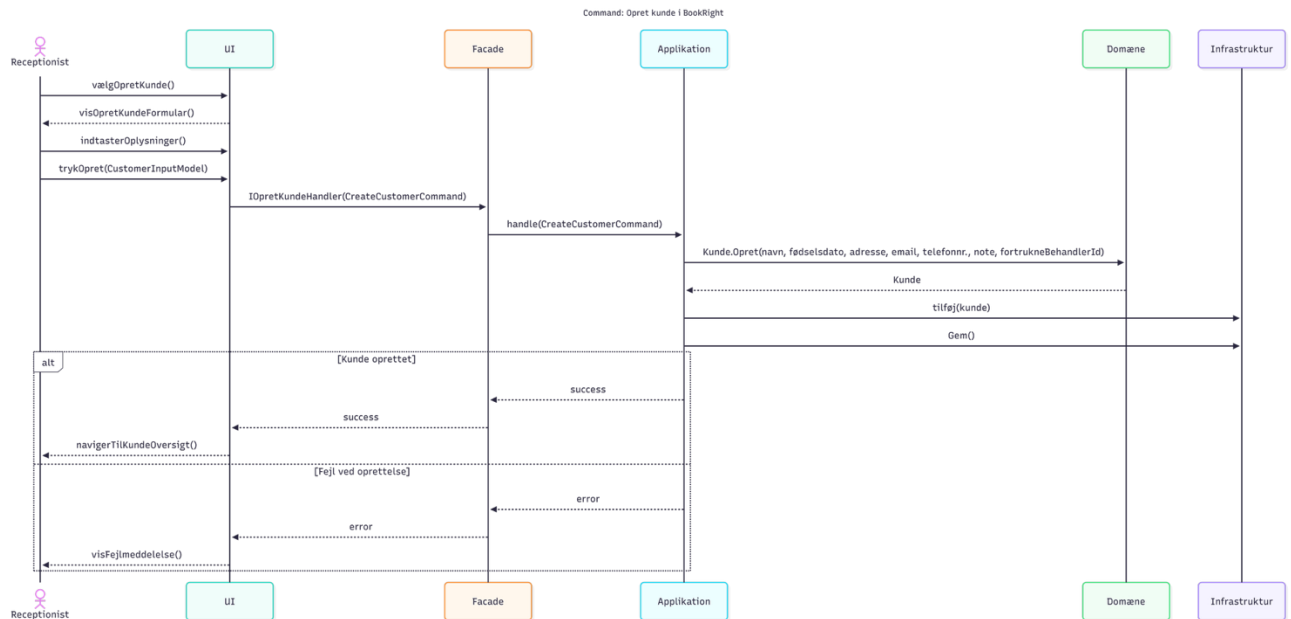
BookRight
C4 Level
4. Code

Kirstine-Amalie, Henrik, Nicolai H. og Christian F



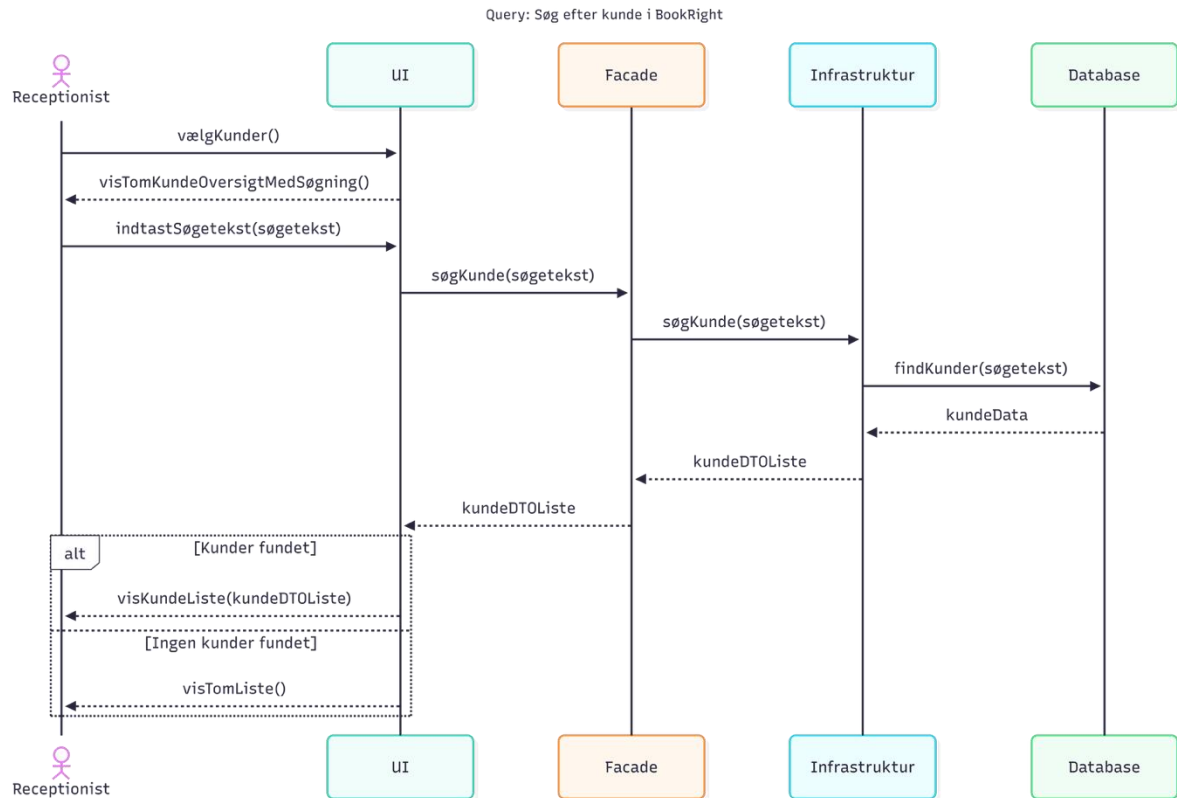
Klassediagrammet viser systemets centrale domæneobjekter og deres relationer. Diagrammet understøtter anvendelsen af DDD ved at synliggøre Booking som central Aggregate Root samt relevante Entities og Value Objects

6.4.3 Sekvensdiagram for Command



Sekvensdiagrammet viser, hvordan en handling, der ændrer systemets tilstand, bevæger sig fra Blazor-brugergrænsefladen gennem Facade- og Use Case-laget til domænelogik og datalagring. Diagrammet illustrerer dermed command-flowet i systemets lagdelte arkitektur.

6.4.4 Sekvensdiagram for Query

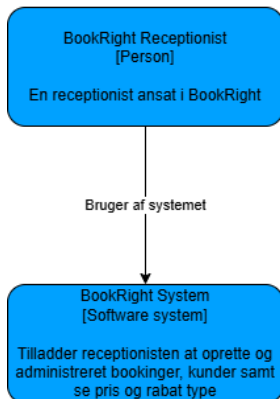


Sekvensdiagrammet viser, hvordan data hentes fra systemet og returneres til Blazor-brugergrænsefladen uden at ændre systemets tilstand. Diagrammet tydeliggør dermed Query-flowet og adskillelsen mellem læsning og ændring af data.

6.5 C4-modellen

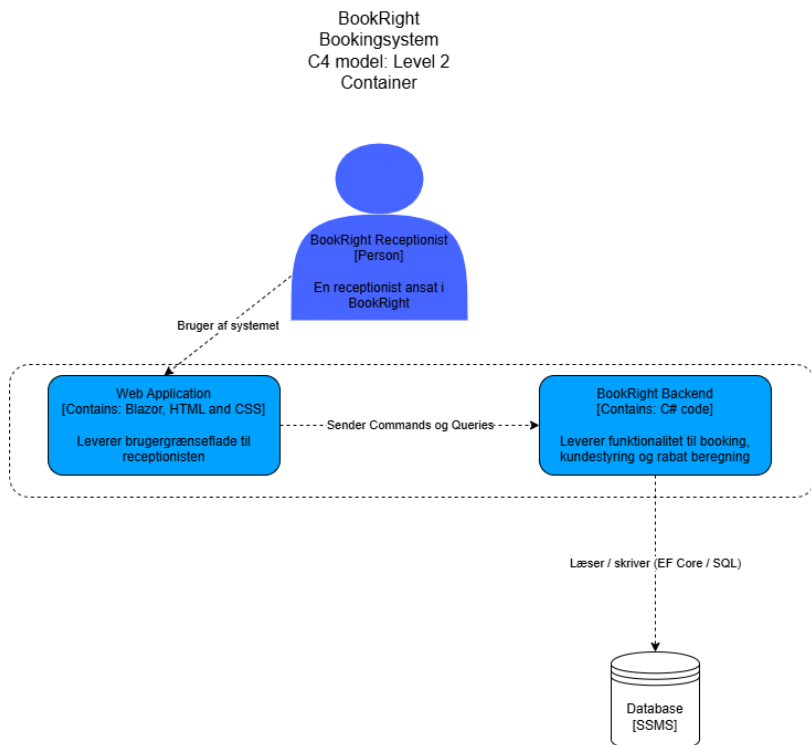
6.5.1 Context-diagram – C1

BookRight
Bookingsystem
C4 model: Level 1
Context



Context-diagrammet viser BookRight-systemet i dets organisatoriske sammenhæng. Diagrammet synliggør, hvordan receptionisten anvender systemet til intern administration, og hvordan løsningen understøtter BookRights drift.

6.5.2 Container-diagram – C2



Container-diagrammet viser systemets overordnede tekniske opdeling og kommunikationen mellem brugergrænseflade, applikationslogik, datatilgang og database. Diagrammet understøtter dokumentationen af løsningens Clean Architecture-baserede struktur.

6.6 Architecture Decision Records

6.6.1 ADR-001: Valg af Clean Architecture med Facade-lag

Titel: Valg af Clean Architecture med Facade-lag

Kontekst: BookRight-systemet indeholder både domæneregler, applikationslogik, datatilgang og en Blazor-baseret brugergrænseflade. For at undgå tæt kobling mellem disse ansvarsområder er der behov for en tydelig lagdeling.

Beslutning: Løsningen struktureres efter Clean Architecture med separate lag for Domain, Use Case/Application, Facade, Infrastructure og UI. Facade-laget anvendes som den eneste kontrakt mellem Blazor-brugergrænsefladen og den bagvedliggende funktionalitet.

Konsekvenser: Arkitekturen skaber tydelig ansvarsfordeling, lavere kobling og bedre mulighed for isoleret test af domæne- og applikationslogik. Til gengæld medfører lagdelingen flere projekter, interfaces og DTO'er, hvilket øger kompleksiteten i løsningens struktur.

6.6.2 ADR-002: Valg af Command Query Separation

Titel: Valg af Command Query Separation (CQS)

Kontekst: Systemet skal både håndtere handlinger, der ændrer data, f.eks. oprettelse og ændring af bookinger, samt forespørgsler, der alene henter data til visning i brugergrænsefladen.

Beslutning: Commands og Queries adskilles efter CQS-princippet. Commands anvendes ved ændringer i systemets tilstand, mens Queries anvendes til læsning og præsentation af data.

Konsekvenser: Adskillelsen giver en mere overskuelig struktur og gør det tydeligere, hvilke funktioner der ændrer data. Det understøtter samtidig testbarhed og sporbarhed. Konsekvensen er, at systemet får flere handlers og interfaces, som skal vedligeholdes.

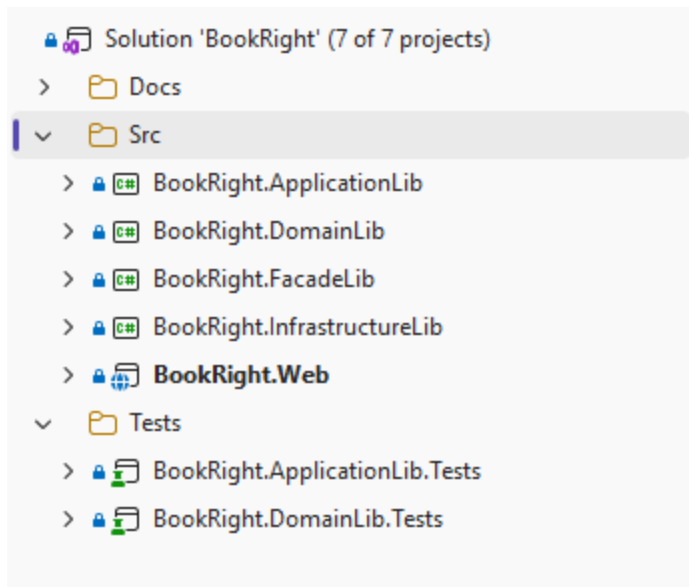
6.6.3 Delkonklusion

De dokumenterede arkitekturbeslutninger understøtter samlet set en struktureret og testbar løsning. Clean Architecture med Facade-lag bidrager til tydelig ansvarsfordeling og reduceret kobling, mens CQS skaber en klar adskillelse mellem handlinger, der ændrer data, og forespørgsler, der henter data. Valgene danner dermed et relevant arkitektonisk grundlag for udviklingen af BookRight-prototypen.

7. Implementering og Programmering I

7.1 Realisering af Clean Architecture i solutionen

Den arkitektoniske lagdeling er realiseret gennem separate .NET-projekter for Domain, Use Case/Application, Facade, Infrastructure og UI. Denne projektstruktur understøtter den valgte Clean Architecture-opbygning og sikrer, at de enkelte lag kan udvikles og vedligeholdes med tydeligt afgrænsede ansvar.



7.2 Implementering af domænemodellen

Implementeringen af domænemodellen tager udgangspunkt i det design, der er beskrevet i afsnit 6.2.

Booking er implementeret som central Aggregate Root med metoder til håndtering af bookings tilstand og centrale forretningsregler. TimeSlot er implementeret som Value Object og indeholder validering af tidsinterval samt logik til kontrol af overlap mellem bookinger.

```
public class Booking : AggregateRoot
{
    8 references
    public Guid TreatmentId { get; private set; }
    11 references
    public Guid TherapistId { get; private set; }
    7 references
    public Guid ClinicId { get; private set; }

    21 references
    public BookingStatus Status { get; private set; }
    16 references
    public TimeSlot Time { get; private set; } = null!;
    4 references
    public decimal Price { get; private set; }

    5 references
    public int ParticipantLimit { get; private set; }
    2 references
    public bool IsActive => Status != BookingStatus.Cancelled;
    5 references
    public DiscountTypes DiscountTypeUsed { get; init; }

    private readonly List<Guid> _participants = new();
    8 references
    public IReadOnlyList<Guid> Participants => _participants.AsReadOnly();

    /// <summary> Creates a Booking with the specified time slot, treatment, therapi ...
    12 references
    public static Booking Create(...).

    /// <summary> Update the Time property to the specified TimeSlot after validatin ...
    3 references
    public void ChangeTime(TimeSlot newTimeSlot)...

    /// <summary> Changes the entity's treatment to the specified identifier.
    5 references
    public void ChangeTreatment(Guid newTreatmentId)...

    /// <summary> Updates the entity's TherapistId to a new therapist after validati ...
    3 references
    public void ChangeTherapist(Guid newTherapistId)...

    /// <summary> Changes the associated clinic to the specified clinic identifier.
    5 references
    public void ChangeClinic(Guid newClinicId)...

public record TimeSlot
{
    14 references
    public DateTime From { get; init; }
    12 references
    public DateTime To { get; init; }

    18 references
    public TimeSlot(DateTime from, DateTime to)
    {
        if (to <= from)
            throw new DomainException(message: "End time must be after start time.");

        From = from;
        To = to;
    }

    1 reference
    public TimeSpan Duration => To - From;

    2 references
    internal bool OverlapsWith(TimeSlot other) => From < other.To && other.From < To;

    // Parameterløs constructor til EF Core
    0 references
    private TimeSlot() { }
}
```

7.3 Implementering af CQS og Facade

CQS- og Facade-strukturen er implementeret gennem separate Commands, Query interfaces og DTO'er. Commands anvendes til handlinger, der ændrer data, mens Queries anvendes til læsning af data. Facade-laget udgør den kontrakt, som UI-laget kommunikerer igennem.

The screenshot displays the Visual Studio interface for a solution named 'BookRight' (7 of 7 projects). The left pane shows the project structure under 'Src', including folders for 'BookRight.ApplicationLib', 'BookRight.DomainLib', 'BookRight.FacadeLib', 'Dependencies', 'Commands', 'Booking', 'Campaigns', 'DTOs', 'Interfaces', 'Clinics', 'Customers', 'Pricing', 'Therapists', 'DTO', 'Queries', 'Bookings', and 'Campaigns'. The right pane shows the code for three interfaces and a DTO:

```
8 references
public interface ICreateBookingHandler
{
    6 references
    Task Handle(CreateBookingCommand command);
}

9 references
public record CreateBookingCommand(
    Guid TreatmentId,
    Guid TherapistId,
    Guid ClinicId,
    DateTime From,
    DateTime To,
    string DiscountTypeUsed,
    Guid? CustomerId = null);

7 references
public interface IBookingQueries
{
    2 references
    Task<BookingDTO?> GetByIdAsync(Guid id);
    3 references
    Task<IReadOnlyList<BookingDTO>> GetAllAsync();
}

10 references
public record BookingDTO(
    Guid BookingId,
    Guid TreatmentId,
    Guid TherapistId,
    Guid ClinicId,
    DateTime From,
    DateTime To,
    decimal Price,
    int ParticipantLimit,
    IReadOnlyList<Guid> Participants,
    string Status,
    string DiscountTypeUsed);
```

7.4 Blazor-brugergrænseflade og Dependency Injection

Brugergrænsefladen udvikles i Blazor og er målrettet systemets eneste brugerrolle: receptionisten. Brugergrænsefladen skal understøtte centrale administrative opgaver, herunder oprettelse og administration af bookinger, registrering af kunder, håndtering af bookingstatus samt visning af relevante oversigter.

Blazor-komponenterne tilgår systemets funktionalitet gennem injicerede Facade-interfaces. Ved hjælp af Dependency Injection afhænger brugergrænsefladen af abstraktioner frem for konkrete implementeringer. Det betyder, at UI-laget ikke selv opretter f.eks. handlers eller Repositories, men modtager de nødvendige afhængigheder gennem systemets konfiguration. Denne tilgang reducerer kobling og understøtter både testbarhed og udskiftelighed.

```
0 references
public static class QueriesDI
{
    1 reference
    public static IServiceCollection AddQueriesDI(this IServiceCollection services)
    {
        services.AddScoped<IBookingQueries, BookingQueryHandlerIMPL>();
        services.AddScoped<ICampaignQueries, CampaignQueryHandlerIMPL>();
        services.AddScoped<IClinicQueries, ClinicQueryHandlerIMPL>();
        services.AddScoped<ICustomerQueries, CustomerQueryHandlerIMPL>();
        services.AddScoped<ITherapistQueries, TherapistQueryHandlerIMPL>();
        services.AddScoped<ITreatmentQueries, TreatmentQueryHandlerIMPL>();
        services.AddScoped<ICertificationTypeQueries, CertificationTypeQueryHandlerIMPL>();

        return services;
    }
}
```

7.5 Designmønstre og SOLID

Projektet anvender designmønstrene Repository, Dependency Injection og Strategy med henblik på at skabe en tydelig og fleksibel systemstruktur. Repository Pattern anvendes til at abstrahere datatilgangen, så Use Case-laget kommunikerer gennem interfaces frem for direkte med databasen. Dependency Injection anvendes til at forbinde interfaces med konkrete implementeringer, f.eks. når handlers modtager Repositories gennem deres Constructor, eller når Blazor-komponenter modtager Facade-interfaces. Strategy Pattern anvendes i forbindelse med pris- og rabatberegning, hvor forskellige rabattetyper kan implementeres som selvstændige strategier.

SOLID-principperne anvendes som retningslinjer for systemets opbygning. Single Responsibility Principle understøttes ved, at handlers, Repositories, Query handlers og domæneklasser har afgrænsede ansvar. Open/Closed Principle kommer til udtryk gennem rabatstrategierne, hvor nye rabattyper kan tilføjes med begrænsede ændringer i den eksisterende beregningslogik. Dependency Inversion Principle understøttes ved, at Use Case- og UI-laget afhænger af interfaces frem for konkrete implementeringer.

```
5 references
✓ public class CreateBookingHandler(
    IBookingRepository bookingRepository,
    ICustomerRepository customerRepository,
    ITherapistRepository therapistRepository,
    IClinicRepository clinicRepository,
    ITreatmentRepository treatmentRepository,
    IBookingCapacityService bookingCapacityService,
    IValidateOverlapService overlapService) : ICreateBookingHandler
{
    6 references
    > async Task ICreateBookingHandler.Handle(CreateBookingCommand command) ...
}
```

7.6 Rabatberegning med Strategy Pattern og parallel evaluering

Rabatberegningen udgør en central forretningsregel i BookRight-systemet, da kunden ved oprettelse af en booking skal tildeles den mest fordelagtige relevante pris. Til dette formål anvendes Strategy Pattern, hvor hver rabattype implementeres som en selvstændig beregningsstrategi. Systemet kan dermed evaluere forskellige rabatmuligheder, f.eks. loyalitetsrabat, fødselsdagsrabat og kampagnerabat, uden at al rabatlogik samles i én kompleks klasse.

Når en booking oprettes, samles de relevante rabatstrategier og afvikles parallelt som CPU-bound tasks. Resultaterne sammenlignes efterfølgende, hvorefter den rabat, der giver kunden den laveste pris, vælges automatisk. Implementeringen forbinder dermed Strategy Pattern med BookRights forretningsmæssige krav om korrekt og automatisk prisberegning.

```
/// <summary>
/// Represents a strategy that calculates discounts for pricing inputs and exposes which DiscountTypes it supports.
/// </summary>
/// <remarks>Implementations should be deterministic and thread-safe where possible. Keep each implementation
/// focused on a single discount behavior, validate inputs, and return a PriceCalculatorResult containing adjusted
/// totals and any applied discount metadata. Avoid external side effects.</remarks>
16 references
public interface IDiscountStrategy
{
    12 references
    DiscountTypes DiscountTypes { get; }
    18 references
    PriceCalculatorResult CalculatePrice(PriceCalculatorInput input);
}
```

```
/// <summary>
/// Applies a birthday-month discount when the booking month matches the customer's birth month and the annual usage
/// limit has not been exceeded.
/// </summary>
/// <remarks>Uses a fixed 25% discount and allows one use per year by default. Implements IDiscountStrategy and
/// returns DiscountTypes.BirthdayMonth in the PriceCalculatorResult.</remarks>
5 references
public class BirthDateDiscountStrategy : IDiscountStrategy
{
    private readonly decimal _discountPercentage;
    private readonly int _maxUsePerYear;

    2 references
    public BirthDateDiscountStrategy()
    {
        _discountPercentage = 25;
        _maxUsePerYear = 1;
    }

    2 references
    public DiscountTypes DiscountTypes => DiscountTypes.BirthdayMonth;

    5 references
    public PriceCalculatorResult CalculatePrice(PriceCalculatorInput input)
    {
        decimal price = input.NormalPrice;
        bool IsApplicable = false;

        if (input.BookingDate.Month == input.CustomerBirthDate.Month &&
            input.NumberOfBirthdayDiscountUsed < _maxUsePerYear)
        {
            price = input.NormalPrice * (1 - _discountPercentage / 100);
            IsApplicable = true;
        }

        return new PriceCalculatorResult(input.NormalPrice, price, DiscountTypes, IsApplicable);
    }
}
```



```

var PriceCalculatorDTO = new PriceCalculatorInput(
    treatment.Price,
    command.BookingDate,
    Customer.BirthDate,
    CustomerHistorySum,
    numberOfBirthdayDiscountUsed,
    allTreatments,
    activeCampaigns);

PriceCalculatorResult? bestDiscount = null;

Parallel.ForEach(_strategies, s =>
{
    var discountResult = s.CalculatePrice(PriceCalculatorDTO);

    lock (_lock)
    {
        if (
            discountResult.IsApplicable == true &&
            (
                bestDiscount == null ||
                discountResult.FinalPrice < bestDiscount.FinalPrice
            )
        )
        {
            bestDiscount = discountResult;
        }
    }
});

if (bestDiscount != null)
    return new PriceCalculatorDTO(bestDiscount.NormalPrice, bestDiscount.FinalPrice, bestDiscount.DiscountType.ToString());

throw new Exceptions.ApplicationException(message: "Error while calculating price");
}

```

7.7 EF Core Code First i implementeringen

Til persistens anvendes Entity Framework Core Code First, hvor databaseadgangen tager udgangspunkt i systemets entiteter og konfigurationer. BookRightDbContext indeholder de centrale datasæt for blandt andet bookinger, kunder, behandlere, klinikker, behandlinger og kampagner. Hvor EF Cores konventioner ikke er tilstrækkelige, anvendes eksplicit konfiguration, f.eks. ved relationer, konvertering af Enum-værdier eller seed data. Den nærmere vurdering af det genererede databaseskema i forhold til normalisering behandles i afsnittet Teknologi I.

```
public class BookRightDbContext : DbContext
{
    public DbSet<Booking> Bookings { get; set; }
    public DbSet<Clinic> Clinics { get; set; }
    public DbSet<Customer> Customers { get; set; }
    public DbSet<Therapist> Therapists { get; set; }
    4 references
    public DbSet<Treatment> Treatments { get; set; }
    5 references
    public DbSet<Campaign> Campaigns { get; set; }
    0 references
    private DbSet<OpeningHour> OpeningHours { get; set; }

    0 references
    public BookRightDbContext(DbContextOptions<BookRightDbContext> options) : base(options) { }

    0 references
    protected override void OnConfiguring(DbContextOptionsBuilder optionsBuilder)
    {
    }

    0 references
    protected override void OnModelCreating(ModelBuilder modelBuilder)
    {
        modelBuilder.ApplyConfigurationsFromAssembly(typeof(BookRightDbContext).Assembly);
    }
}
```

```
0 references
internal class TreatmentConfiguration : IEntityTypeConfiguration<Treatment>
{
    0 references
    public void Configure(EntityTypeBuilder<Treatment> builder)
    {
        builder.Property(t => t.CertificationRequired)
            .HasConversion<string>();

        builder.HasData(
            new { Id = Guid.Parse(input: "68a14b26-15d6-4831-a876-221b01260c6b"),
                  Name = "Acupuncture 45",
                  Price = 550m,
                  Duration = TimeSpan.FromMinutes(45),
                  CertificationRequired = CertificationTypes.Acupuncture,
                  MaxParticipants = 1 },
            new { Id = Guid.Parse(input: "f748fc78-c660-4de0-ala8-f7c0b0930f98"),
                  Name = "Dietary Guidance follow-up",
                  Price = 450m,
                  Duration = TimeSpan.FromMinutes(30),
                  CertificationRequired = CertificationTypes.Dietary,
                  MaxParticipants = 1 },
            new { Id = Guid.Parse(input: "79b16d43-7475-4409-bd12-ddd0e2067863"),
                  Name = "Dietary Guidance first time",
                  Price = 799m, Duration = TimeSpan.FromMinutes(60),
                  CertificationRequired = CertificationTypes.Dietary,
                  MaxParticipants = 1 },
        );
    }
}
```

7.8 Asynkrone databasekald og parallel udførelse

I løsningen anvendes `async/await` ved databaseoperationer gennem Repositories og Query handlers, så databasekald kan udføres asynkront. Derudover anvendes parallel udførelse i forbindelse med rabatberegningen, hvor relevante rabatstrategier evalueres med henblik på at identificere kundens mest fordelagtige pris. Den teknologiske redegørelse for I/O-bound og CPU-bound arbejde fremgår af afsnittet Teknologi I.

```
6 references
async Task ICreateBookingHandler.Handle(CreateBookingCommand command)
{
    var therapist = await therapistRepository.GetByIdAsync(command.TherapistId)
    ?? throw new NotFoundException(message: "Therapist could not be found.");

    var treatment = await treatmentRepository.GetByIdAsync(command.TreatmentId)
    ?? throw new NotFoundException(message: "Treatment could not be found.");

    if (therapist.CertificationTypes.Contains(treatment.CertificationRequired) == false)
        throw new Exceptions.ApplicationException(message: "Therapist is not qualified for this treatment.");
}
```

7.9 Delkonklusion

Programmeringsarbejdet i BookRight-projektet tager udgangspunkt i en lagdelt og udvidelsesbar prototypearkitektur. Clean Architecture skaber en tydelig adskillelse mellem domænelogik, applikations flow, kontrakter, infrastruktur og brugergrænseflade, mens DDD bidrager til, at centrale forretningsregler modelleres direkte i systemets domæne. CQS og Facade-laget understøtter en struktureret kommunikation mellem Blazor-brugergrænsefladen og systemets funktionalitet, mens designmønstrene Repository, Dependency Injection og Strategy bidrager til lavere kobling og større fleksibilitet.

Rabatberegningen viser, hvordan en konkret forretningsregel kan implementeres gennem separate strategier og parallel evaluering, mens anvendelsen af `async/await` ved databasekald dokumenterer systemets asynkrone datatilgang. Samlet set understøtter programmeringsvalgene udviklingen af en prototype, der er struktureret og tilpasset BookRights centrale administrative behov.

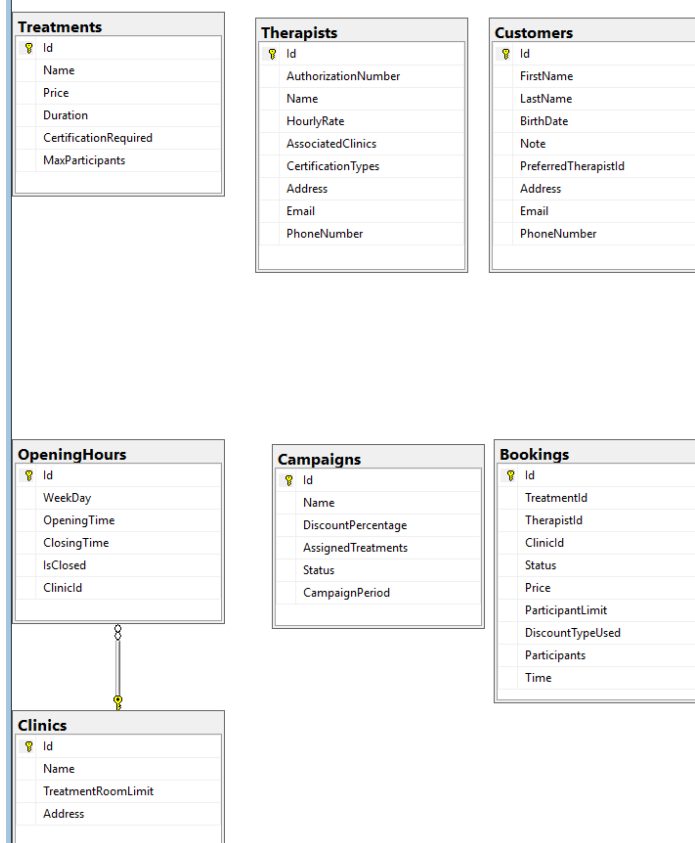
8. Teknologi I

8.1 Code First og normalisering

I BookRight-systemet anvendes Entity Framework Core Code First, hvor databaseskemaet genereres med udgangspunkt i domænemodellen. Centrale domæneobjekter som bookinger, kunder, behandlere, klinikker, behandlinger og kampagner er repræsenteret som selvstændige tabeller. Hver tabel identificeres gennem en primary key, mens relationer mellem tabellerne etableres gennem foreign keys, f.eks. når en booking refererer til en behandling, en behandler og en klinik. Dette understøtter referentiel integritet og reducerer behovet for at gentage oplysninger i bookingtabellen.

Databaseskemaet vurderes at opfylde 1. normalform (1NF), idet attributterne lagres som atomare værdier, og eventuelle relationer eller samlinger håndteres relationelt. 2. normalform (2NF) vurderes opfyldt, da tabellernes ikke-nøglefelter afhænger af den tilhørende primary key. 3. normalform (3NF) understøttes ved, at oplysninger om f.eks. kunder, behandlere, klinikker og behandlinger er placeret i selvstændige tabeller og forbindes gennem foreign keys frem for at blive gentaget i bookingdata.

Som illustreret bidrager anvendelsen af primary keys, foreign keys og separate entitetstabeller til en relationel databasestruktur, der reducerer redundans og understøtter dataintegritet i BookRight-systemet.



8.2 Proces, tråde og I/O

I relation til systemets drift er begreberne proces, tråd og I/O relevante. Når BookRight-systemet afvikles, kører applikationen som en proces, der indeholder de ressourcer, der er nødvendige for programmets udførelse. Inden for processen anvendes tråde til at udføre systemets opgaver. I/O opstår blandt andet, når systemet kommunikerer med databasen for at hente eller gemme bookinger, kunder og øvrige data. Da databaseoperationer primært indebærer ventetid på en ekstern ressource, anvendes `async/await`, så den udførende tråd ikke blokeres unødvendigt, mens systemet afventer svar fra databasen.

8.3 I/O-bound og CPU-bound parallelisme

Der skelnes i projektet mellem I/O-bound og CPU-bound arbejde. I/O-bound opgaver, eksempelvis databasekald gennem Repositories og Query handlers, er afhængige af ventetid og håndteres derfor asynkront for at forbedre systemets responsivitet og ressourceudnyttelse. CPU-bound opgaver kræver derimod aktiv beregningskraft, f.eks. når flere rabatstrategier evalueres for at identificere den mest fordelagtige pris for kunden. Ved parallel udførelse af CPU-bound beregninger kan

operativsystemets trådplanlægning (thread scheduling) fordele arbejdet mellem de tilgængelige processorkerner. I prototypen anvendes paralleliseringen målrettet i rabatberegningen, hvor flere relevante strategier evalueres for at vælge den bedste pris for kunden.

8.4 Delkonklusion

Samlet set dokumenterer afsnittet sammenhængen mellem systemets tekniske implementering og de teknologiske principper, der ligger til grund for løsningen. EF Core Code First skaber sammenhæng mellem domænemodel og database, mens vurderingen af det genererede skema tydeliggør betydningen af korrekt mapping og normalisering. Anvendelsen af asynkrone databasekald demonstrerer forståelse for I/O-bound arbejde, mens parallel evaluering af rabatstrategier illustrerer CPU-bound arbejde og operativsystemets håndtering af tråde. Dermed behandles både databaseteknologiske og driftsmæssige perspektiver i relation til BookRight-systemet.

9. Test og kvalitetssikring

9.1 Teststrategi

Teststrategien i BookRight-projektet tager udgangspunkt i, at centrale forretningsregler og Use Cases skal kunne verificeres isoleret. Domain-laget testes gennem unit tests af domæneregler og Value Objects, mens Use Case-laget testes med Moq, så afhængigheder til Repositories og services kan simuleres. Testningen har været integreret i udviklingsprocessen og anvendes til at understøtte både kvalitetssikring og refaktoreringer.

9.2 Unit tests for Domain-laget

Domain-laget indeholder systemets centrale forretningsregler og er derfor et væsentligt fokusområde for automatiseret test. Der er udarbejdet unit tests, som verificerer domænelogikken uafhængigt af database og brugergrænseflade. Testene omfatter blandt andet validering af TimeSlot, ændring af bookingstatus samt regler for ændringer og håndtering af bookinger.

Denne test type er relevant, fordi fejl i domænereglerne kan føre til ugyldige bookinger eller inkonsistente data. Ved at teste reglerne isoleret kan vi dokumentere, at centrale dele af forretningslogikken fungerer som forventet.

```

0 references
public class CustomerTests
{
    1 reference
    private static string FirstName => "Poul";
    1 reference
    private static string LastName => "Pedersen";
    1 reference
    private static DateOnly BirthDate => new DateOnly(year: 1965, month: 6, day: 18);
    1 reference
    private static string Note => "Peanutbutter-allergy";
    1 reference
    private static Address Address => new Address(street: "Test Avenue 21", postalCode: "1234", city: "Testville");
    1 reference
    private static Email Email => new Email(emailAddress: "PoulP@testmail.com");
    1 reference
    private static PhoneNumber PhoneNumber => new PhoneNumber(number: "87654321");

    16 references
    private static Customer CreateCustomerWithValidData(...)

    // -----
    // 1. Create tests (Creating a Customer)
    // -----

    [Fact]
    0 references
    public void Create_WithInvalidFirstName_CastDomainException()
    {
        //Arrange
        string firstName = "";

        //Act & Assert
        Assert.Throws<DomainException>(() => CreateCustomerWithValidData(firstName: firstName));
    }
}

```

9.3 Unit tests for Use Cases med Moq

Use Case-laget testes med anvendelse af Moq, så afhængigheder til Repositories og øvrige services kan simuleres. Dette gør det muligt at teste, om en handler udfører det korrekte applikations flow, uden at testen kræver adgang til en virkelig database. Eksempelvis kan en test verificere, at en booking gemmes ved gyldige input, eller at en relevant Exception kastes, hvis behandleren ikke kan udføre behandlingen, hvis der opstår overlap, eller hvis klinikkens kapacitet er overskredet.

Brugen af Moq understøtter isolerede og reproducerbare tests, idet vi selv kan styre, hvilke data Repositories returnerer i den enkelte testsituation. Derved dokumenteres både korrekt anvendelse af afhængigheder og korrekt håndtering af de forretningsmæssige scenarier i applikationslaget.

```

0 references
public class CustomerArrivedTests
{
    1 reference
    private static TimeSlot CreateTimeSlot(int fromHour, int toHour) {...}

    1 reference
    private static Booking CreateBooking() {...}

    // -----
    // 1. Handle tests (Registering customer arrival through Application)
    // -----

    [Fact]
    0 references
    public async Task Handle_GivenValidBookingId_CallsSave()
    {
        // Arrange
        Booking booking = CreateBooking();

        var bookingRepositoryMock = new Mock<IBookingRepository>();

        bookingRepositoryMock
            .Setup(r => r.GetByIdAsync(booking.Id))
            .ReturnsAsync(booking);

        ICustomerArrivedHandler handler = new CustomerArrivedHandler(bookingRepositoryMock.Object);

        var command = new CustomerArrivedCommand(booking.Id);

        // Act
        await handler.Handle(command);

        // Assert
        bookingRepositoryMock.Verify(r => r.SaveAsync(), Times.Once);
    }
}

```

9.4 Testresultater

Test	Duration	Traits
✓ BookRight.ApplicationLib.Tests (61)	3,5 sec	
> ✓ BookRight.ApplicationLib.Tests.Bookings (27)	1,8 sec	
> ✓ BookRight.ApplicationLib.Tests.Clinics (4)	481 ms	
> ✓ BookRight.ApplicationLib.Tests.Customers (13)	511 ms	
> ✓ BookRight.ApplicationLib.Tests.Services (6)	276 ms	
> ✓ BookRight.ApplicationLib.Tests.Therapists (11)	491 ms	
✓ BookRight.DomainLib.Tests (101)	410 ms	
> ✓ BookRight.DomainLib.Tests.DiscountStrategies (21)	115 ms	
> ✓ BookRight.DomainLib.Tests.Entities (64)	140 ms	
> ✓ BookRight.DomainLib.Tests.Services (5)	56 ms	
> ✓ BookRight.DomainLib.Tests.ValueObjects (11)	99 ms	

10. Dokumentation af processen

Projektet er gennemført gennem seks sprints, benævnt Sprint 0 til Sprint 5. Sprintene er anvendt som en del af vores modificerede Scrum-proces, hvor udviklingsarbejdet løbende er blevet planlagt, gennemført, evalueret og tilpasset. Rapportskrivning og øvrig dokumentation er foregået sideløbende gennem hele projektperioden, mens de enkelte sprints har haft forskellige tekniske og procesmæssige fokusområder. De følgende sprintbeskrivelser er udarbejdet på baggrund af vores Daily Stand-ups, Sprint Reviews, Sprint Retrospectives samt det gennemførte udviklingsarbejde.

10.1 Sprint 0 - Projektgrundlag og planlægning

Sprint 0 havde til formål at etablere projektets organisatoriske, metodiske og tekniske grundlag. Vi udarbejdede en gruppekontrakt med fælles retningslinjer for samarbejde, kommunikation, deadlines og håndtering af fravær. Derudover blev der fastlagt kodestandarder med henblik på at sikre ensartet og vedligeholdelsesvenlig kode på tværs af vores arbejde.

I sprintet blev projektets rapportstruktur etableret, og der blev arbejdet med indledende analyse- og designdokumentation, herunder relevante diagrammer. Vi strukturerede desuden arbejdet i backloggen gennem Epics, Features, User Stories og Tasks. Sprint 0 skabte dermed det fælles fundament for den efterfølgende udvikling af BookRight-systemet.

10.2 Sprint 1 - Domænemodel og centrale forretningsregler

Sprint 1 havde fokus på etableringen af systemets Domain-lag. Her blev centrale domæneobjekter som Booking, Clinic, Customer, Therapist og Treatment implementeret med henblik på at modellere BookRights væsentligste forretningsbegreber og regler. Der blev desuden arbejdet med centrale Value Objects og domænemetoder, så blandt andet tidsrum, bookingstatus og ændringer i bookinger kunne håndteres i domænet.

Som en del af sprintets kvalitetssikring blev der udarbejdet unit tests for Domain-laget, så centrale forretningsregler kunne verificeres uafhængigt af database og brugergrænseflade. Sprint Review viste, at det grundlæggende domænelag var etableret, mens Sprint Retrospective identificerede udfordringer i vores anvendelse af GitHub og i den interne koordinering. Særligt arbejdet med



branches, commits og sammenfletning af kode medførte ekstra tidsforbrug. Erfaringerne førte derfor til øget fokus på tydelig koordinering og tidligere dialog om tekniske udfordringer i de efterfølgende sprints.

10.3 Sprint 2 - Use Cases, Facade og kvalitetssikring

Sprint 2 havde fokus på udviklingen af systemets Use Case- og Facade-lag. Der blev arbejdet med handlers til centrale funktioner i booking flowet, herunder oprettelse af bookinger, registrering af kunders ankomst og håndtering af gennemførte bookinger. Derudover blev der udviklet funktionalitet relateret til kunder og behandlere samt tilhørende tests, så applikationslogikken kunne verificeres isoleret.

Sprintets Review viste behov for yderligere kvalitetssikring af både domænemodel og teststruktur. Som følge heraf blev dele af løsningen refaktoreret, og testopsætningen blev forbedret ved at anvende mere isolerede mock-objekter, hvilket gjorde testene mere pålidelige og lettere at vedligeholde. På baggrund af erfaringerne med versionsstyring fra det foregående sprint valgte vi desuden at samle udviklingsarbejdet i en fælles DEV-branch. Formålet var at forenkle koordineringen og skabe et mere fælles udviklingsgrundlag for den videre implementering.

10.4 Sprint 3 - Infrastruktur, rabatberegning og forbedringer

Sprint 3 havde fokus på at videreføre backend-udviklingen og forberede overgangen til brugergrænsefladen. Der blev arbejdet med Infrastructure-laget og sammenkoblingen mellem systemets lag, så dataadgang og den bagvedliggende funktionalitet kunne understøtte den kommende Blazor-brugergrænseflade.

Derudover blev der arbejdet med rabatberegningen og anvendelsen af Strategy Pattern, som skal understøtte systemets krav om automatisk beregning af den mest fordelagtige pris for kunden. Sprintet var mindre omfattende end de foregående, men bidrog til at konsolidere backend-løsningen og skabe grundlag for UI-arbejdet. I Sprint Retrospective blev der desuden reflekteret over betydningen af tydelige afgrænsninger og realistisk prioritering af funktionalitet.



Sprintet blev påvirket af reduceret kapacitet i gruppen, hvilket nødvendiggjorde omfordeling af enkelte opgaver. Retrospective tydeliggjorde derfor behovet for realistisk afgrænsning og prioritering af den funktionalitet, der var mest central for den samlede prototype.

10.5 Sprint 4 - Brugergrænseflade i Blazor

Sprint 4 havde primært fokus på udviklingen af systemets brugergrænseflade i Blazor. Formålet var at omsætte den allerede udviklede domæne- og applikationslogik til en anvendelig løsning for receptionistens daglige arbejdsopgaver.

Der blev arbejdet med opbygning af UI-komponenter samt integration mellem brugergrænsefladen og de bagvedliggende funktioner gennem Facade-laget. Arbejdet tog udgangspunkt i, at receptionisten er systemets primære bruger, og at brugergrænsefladen derfor skal understøtte centrale handlinger som bookinghåndtering, visning af relevante oplysninger og administration af kunder og behandlinger. Sprintet markerede dermed overgangen fra primært backend-orienteret udvikling til en mere samlet og demonstrerbar prototype.

10.6 Sprint 5 - Færdiggørelse, kvalitetssikring og rapportfærdiggørelse

Sprint 5 havde til formål at samle og færdiggøre projektets tekniske og skriftlige leverancer. Der blev arbejdet med gennemgang og finpudsning af den implementerede funktionalitet, herunder rettelser, kvalitetssikring og kontrol af sammenhængen mellem systemets lag. Derudover blev tests og builds gennemgået med henblik på at sikre, at prototypens centrale funktioner kunne dokumenteres og demonstreres.

Rapportarbejdet, som havde fundet sted løbende gennem hele projektperioden, blev i dette sprint samlet og færdigbearbejdet. Der blev blandt andet arbejdet med dokumentation af proces, krav, arkitektur, forretningsanalyse, teknologiske valg, tests og refleksioner over projektets udvikling. Sprintet havde således fokus på at sikre sammenhæng mellem krav, design, implementering, test og den endelige formidling af projektet.

I de afsluttende sprints blev der lagt øget vægt på sammenhæng mellem implementering, tests og dokumentation. Erfaringerne fra de tidligere sprints blev anvendt til at prioritere kvalitetssikring, rettelser og færdiggørelse af de vigtigste bruger flows.

10.7 Procesmæssige udfordringer og læring fra retrospectives

Gennem projektets retrospectives identificerede vi flere udfordringer, som havde betydning for både samarbejdet og udviklingsprocessen. I de tidlige sprints opstod der særligt usikkerhed omkring anvendelsen af GitHub, herunder branches, commits, pushes og sammenfletning af kode. Dette medførte i enkelte tilfælde ekstra tidsforbrug og behov for efterfølgende rettelser. Som konsekvens valgte vi senere i forløbet at samle udviklingsarbejdet i en fælles DEV-branch med henblik på at forenkle koordineringen og skabe et mere fælles udviklingsgrundlag.

Der blev ligeledes identificeret udfordringer i relation til intern kommunikation og forventningsafstemning. Tekniske udfordringer blev i enkelte tilfælde undersøgt individuelt i længere tid end nødvendigt, før de blev synliggjort for os. Vi arbejdede derfor løbende med at skabe større åbenhed omkring tvivl, fejl og behov for faglig sparring. I de efterfølgende sprints bidrog dette til, at spørgsmål i højere grad blev afklaret tidligere i arbejdsprocessen.

Kvalitetssikring udgjorde desuden et centralt læringspunkt. I forbindelse med gennemgang af kode og tests blev der identificeret fejl og uhensigtsmæssigheder, som krævede refaktoreringer. Blandt andet blev testopsætninger ændret, så tests i højere grad anvendte isolerede mock-objekter frem for delte afhængigheder. Erfaringen viste, at løbende egenkontrol, test og kodegennemgang er væsentlige for at reducere efterfølgende rettelsesarbejde og sikre en mere stabil udviklingsproces.

Projektforløbet blev desuden påvirket af perioder med reduceret kapacitet i gruppen, hvilket medførte behov for omfordeling af enkelte opgaver. Dette tydeliggjorde betydningen af realistisk afgrænsning og prioritering af de funktioner, der var mest centrale for den samlede prototype.

Samlet set viser sprintforløbet en gradvis udvikling fra etablering af projektgrundlag og domænemodel til implementering af applikationslogik, infrastruktur, brugergrænseflade og afsluttende kvalitetssikring. Den modificerede Scrum-proces har gjort det muligt løbende at evaluere arbejdsformen og foretage konkrete justeringer i relation til branch strategi, kommunikation, kvalitetssikring og arbejdsfordeling. Udfordringerne betragtes derfor ikke alene som problemer i forløbet, men som erfaringer, der har bidraget til at kvalificere både vores proces og udviklingen af BookRight-prototypen.

11. Perspektivering

BookRight-prototypen adresserer centrale udfordringer ved virksomhedens manuelle bookingproces ved at skabe grundlag for mere struktureret bookinghåndtering, validering og prisberegning.

Løsningen udgør dog ikke en fuldt færdig driftsløsning, og en videreudvikling bør derfor fokusere på de funktioner, der kan styrke systemets anvendelighed i den daglige drift.

Det vil blandt andet være relevant at videreudvikle systemet med et samlet kalender- og belægningsoverblik, så receptionisten hurtigt kan danne sig et overblik over bookinger, behandlere og klinikkapacitet. Rapportering og eksport af omsætningsdata vil ligeledes kunne give ledelsen et bedre grundlag for opfølgning og beslutningstagning. Derudover kan løsningen på længere sigt udvides med funktioner som fakturering, betaling og mere omfattende statusstyring af bookinger.

På det tekniske niveau bør en videreudvikling desuden omfatte en mere målrettet datatilgang, hvor relevante oplysninger filtreres gennem queries frem for at blive hentet bredt og behandlet i brugergrænsefladen. Det vil kunne forbedre både ydeevne og skalerbarhed, særligt hvis BookRight udvider med flere klinikker og et større kundegrundlag. Databasestrukturen bør desuden videreudvikles, så samlinger, relationer og sammensatte værdier mappes mere hensigtsmæssigt og understøtter en mere normaliseret og robust datamodel. En løsning i egentlig drift vil samtidig kræve yderligere fokus på datasikkerhed og afprøvning med faktiske brugere.

Projektforløbet har også givet væsentlige erfaringer i relation til samarbejde og udviklingsproces. Den modificerede Scrum-proces har bidraget til løbende planlægning, evaluering og tilpasning af arbejdet. Gruppekompakt, kodestandarder, tests og regelmæssige reviews har skabt et fælles grundlag for udviklingen og bidraget til kvalitetssikring af løsningen.

Samtidig opstod der udfordringer med versionsstyring, intern kommunikation og perioder med reduceret kapacitet. Særligt arbejdet med branches og sammenfletning af kode medførte ekstra tidsforbrug, hvilket førte til en tilpasning af branchestrategien og øget fokus på koordinering. Erfaringerne viser, at tekniske udfordringer bør synliggøres tidligt, så vi kan løse problemer i fællesskab frem for at bruge unødigt tid individuelt.



I et fremtidigt projekt vil det derfor være hensigtsmæssigt at fastlægge branchestrategi, code review-proces og fælles definition af færdige opgaver allerede ved projektets begyndelse. Derudover bør dokumentation af sprintresultater og beslutninger foretages mere systematisk gennem hele forløbet. Samlet set har projektet givet erfaring med både udvikling af en lagdelt softwareprototype og betydningen af tydelig kommunikation, realistisk prioritering og løbende kvalitetssikring i et gruppebaseret udviklingsprojekt.

12. Konklusion

Projektets formål har været at udvikle en prototype af et internt administrationssystem, der kan understøtte BookRights bookingarbejde og reducere udfordringerne ved virksomhedens nuværende manuelle proces. Forretningsanalysen viser, at den eksisterende arbejdsgang med telefonbooking, regneark og håndskrevne noter skaber risiko for dobbeltbookinger, manglende overblik og ineffektiv administration. Der er derfor et tydeligt behov for en mere samlet og systemunderstøttet løsning.

På denne baggrund er der udviklet en prototype med fokus på centrale administrative funktioner, herunder bookinghåndtering, kundedata, validering af overlap og kapacitet samt pris- og rabatberegning. Løsningen er struktureret efter Clean Architecture med separate lag for Domain, Use Case/Application, Facade, Infrastructure og UI.DDD, CQS, Repository Pattern, Dependency Injection og Strategy Pattern bidrager til en løsning, hvor forretningsregler, datatilgang og brugergrænseflade er tydeligt adskilt. Dette understøtter både testbarhed og mulighed for videreudvikling.

Projektet har desuden dokumenteret centrale teknologiske overvejelser. EF Core Code First skaber sammenhæng mellem domænemodel og database, mens vurderingen af det genererede databaseskema har synliggjort relevante forbedringsmuligheder i relation til normalisering og datamodellering. Anvendelsen af asynkrone databasekald og parallel evaluering i rabatberegningen demonstrerer samtidig forståelse for forskellen mellem I/O-bound og CPU-bound arbejde.

Kvalitetssikringen er understøttet gennem unit tests af Domain-laget og Use Cases med Moq, hvilket giver grundlag for at verificere centrale forretningsregler og applikationsflows isoleret. Den



modificerede Scrum-proces har samtidig bidraget til løbende planlægning, evaluering og tilpasning af projektarbejdet.

Samlet set viser projektet, hvordan en struktureret og testbar softwarearkitektur kan danne grundlag for en digital løsning, der adresserer BookRights centrale administrative udfordringer. Prototypen udgør ikke en fuldt færdig driftsløsning, men demonstrerer et relevant fundament for mere effektiv bookingadministration, færre fejl og bedre grundlag for videreudvikling af virksomhedens digitale arbejdsgange.

13. Litteraturliste

<https://chatgpt.com>

<https://ucl.itslearning.com> - Undervisningsmateriale.

14. Bilag

Gruppekontrakt:

Gruppekontrakt

Aftaler:

Mødetidspunkt kl. 09, eller efter aftale.

Møder til tiden. Informér om forsinkelser.

Møder ind til Daily scrum.

Vi har en Team Leader/Scrum Master som er: Henrik

Samlet valg af IDE: Visual Studio 2026

Teambuilding (gaming, padel, by osv.).

Vi overholder vores aftaler.

Kodestandard:

Vi koder på engelsk.

Standarder for lag:

- Generelt
 - o C# standarder for navngivning af klasser, metoder, interfaces osv.
 - o Class Library navngives altid med Software navnet først og Lib til sidst
 - F.eks. Application.DomainLib
- Domæne laget
 - o Ingen yderligere bemærkning
- Applikation laget
 - o Use cases betegnes og skrives altid som handlers
- Infrastruktur laget
 - o Queries implementeringer navngives med "IMPL" til slut
- Facade laget
 - o Ingen yderligere bemærkning

Alle klasser skal kun have 1 namespace og dette skal sikres med ny formatering.

Metoder skal have kommentar.

Klasser skal have kommentar.

Større kodelystykker, som ikke er åbenlyse, skal have kommentarer.

Vi skriver vores egen kode. Hvis vi går udenfor pensum, laver vi ekstra opsamling/kodeforklaring.

Git:

Individuelle udviklings-branches.

- Disse branches navngives efterfølgende måde:
 - o Feature: feature
 - o Bug Fixes: bugfix
 - o Design Updates: design
 - o Refactoring: refactor
 - o Hotfixes: hotfix
 - o Documentation: doc
- Foruden dette navngives det også med Sags nummeret fra Dev Ops

f.eks. Feature-DevOps-110-Add-Costumer-Domain

Review policies:

- Min. 1 peer review før pull request til 'main'.
- Team Leader/Scrum Master (eller plenum) review før push til 'main'.

Ambitionsniveau:

Ligelig fordeling af opgaver.

Det er ikke pinligt at spørge om hjælp.

Weekender fri som standard. Ændres efter behov.

Jævnlig opsamling af gruppearbejdet.

Ved overtrædelse af gruppekontrakten:

- Ved udeblivelse fra aftalte gruppemøder og/eller gruppearbejde, bliver der ikke taget højde for fraværet.
- Hvis efter 2 gange ikke overholder deadlines for opgaver, gives mundtligt advarsel.
- Hvis man ikke er kontaktbar eller begrundet fravær, sendes statusbesked til underviserne, samt gives mundtlig advarsel i gruppen.
- Ved tilstrækkelige advarsler, kan gruppemedlem overvejes til udsmidning.
 - o Afgøres i gruppen.

Underskrifter:

Henrik: Henrik Hansen

Christian: Christi Kastedt

Nicolai: Nicolai Heby

Kirstine: Kirstine A.K. Olsen

Low-fi:



1

(X)

kundetilf _____

Behandlingstype ↓

Behandler ↓

Klinik ↓

Book

High-fi:

BookRight	Klinik ▼	Behandlingstype ▼	Behandler ▼
Kalender			
Kunder			
Rapporter			

BookRight	Klinik ▼	Behandlingstype ▼	Behandler ▼
Kalender			
Kunder			
Rapporter			

Fornavn _____

Efternavn _____

Adresse _____

Postnr. _____ **By** _____

Tlf. nr. _____ **Fødselsdato** _____

Email _____

Note

Gem

BookRight	Klinik ▼	Behandlingstype ▼	Behandler ▼						
Kalender	Man. 20/4	Tirs. 21/4	Ons. 22/4	Tors. 23/4	Fre. 24/4	Lør. 25/4	Sen. 26/4	Dag	Uge
Kunder	07:00-08:00								
Rapporter	08:00-09:00								
	10:00-12:00								

BookRight

Kalender

Kunder

Rapporter

Booking

Klinik ▾

Behandlingstype ▾

Behandler ▾

Kundetlf.

Behandlingstype ▾

Behandler ▾

Klinik ▾

Dato

Start tidspunkt ▾

Beregn pris

Book